UNICID — UNIVERSIDADE CIDADE DE SÃO PAULO
ENGENHARIA DE SOFTWARE
Grupo: 5

PATA FELIZ

São Paulo
2026

UNICID — UNIVERSIDADE CIDADE DE SÃO PAULO
ENGENHARIA DE SOFTWARE
Grupo: 5

PATA FELIZ

Trabalho apresentado como requisito para avaliação da disciplina de Modelagem de Dados, do curso de Engenharia de Software, com o objetivo de desenvolver um banco de dados para o sistema de gestão do Pet Shop Pata Feliz, abordando requisitos funcionais e não funcionais, regras de negócio, modelagem de dados, implementação SQL e consultas gerenciais, visando à organização e ao controle das informações da empresa.

São Paulo
2026

SUMÁRIO

1 INTRODUÇÃO……..……1
2 APRESENTAÇÃO DA EMPRESA…….....2
3 PROBLEMA E JUSTIFICATIVA 3
4 OBJETIVOS DO SISTEMA 4
5 REQUISITOS DO SISTEMA 5
6 MODELAGEM CONCEITUAL E DER 6
7 DICIONÁRIO DE DADOS 7
8 DIÁRIOS DE BORDO E DECISÕES DE MODELAGEM 8
9 MODELAGEM LÓGICA 9
10 NORMALIZAÇÃO 10
11 IMPLEMENTAÇÃO SQL 11
12 DADOS PARA TESTES 12
13 CONSULTAS GERENCIAIS E AUDITORIA 13
14 FLUXOGRAMAS 14
15 CONTROLE FINANCEIRO E CONCILIAÇÃO 15
16 INOVAÇÃO E VALOR AGREGADO 16
17 BI E INTELIGÊNCIA ARTIFICIAL 17
18 CONCLUSÃO 18
19 REFERÊNCIAS E ANEXOS 19
20 CHECKLIST FINAL DO GRUPO 20

INTRODUÇÃO

O presente trabalho tem como objetivo apresentar o desenvolvimento de um banco de dados para o sistema de gestão do Pet Shop Pata Feliz, buscando melhorar a organização, o armazenamento e o controle das informações relacionadas às atividades da empresa.

O projeto foi desenvolvido com base nos conhecimentos adquiridos na disciplina de Modelagem de Dados, envolvendo a definição dos requisitos funcionais e não funcionais, as regras de negócio, a elaboração do Diagrama Entidade-Relacionamento (DER), a construção do modelo lógico e a criação do dicionário de dados.

Além disso, o trabalho apresenta a implementação do banco de dados utilizando a linguagem SQL, permitindo o gerenciamento de clientes, animais, funcionários, produtos, serviços, agendamentos, vendas, estoque e movimentações financeiras. Dessa forma, o projeto busca demonstrar a importância de um banco de dados bem estruturado para garantir a integridade das informações e contribuir para uma gestão mais eficiente.

APRESENTAÇÃO DA EMPRESA

O Pet Shop Pata Feliz é uma empresa do ramo de produtos e serviços para animais de estimação, que tem como principal objetivo oferecer cuidados voltados à saúde, higiene e bem-estar dos animais. A empresa busca proporcionar um atendimento de qualidade, garantindo praticidade, segurança e satisfação aos seus clientes.

A empresa oferece diversos produtos e serviços, incluindo a venda de alimentos, acessórios, produtos de higiene e agendamento de atendimentos. Além disso, realiza atividades administrativas relacionadas ao cadastro de clientes, animais, funcionários e fornecedores, assim como ao controle de estoque, compras, vendas e movimentações financeiras.

Com o objetivo de melhorar a organização e a eficiência de suas atividades, o Pet Shop Pata Feliz busca utilizar um sistema de gestão integrado a um banco de dados. Essa solução permite centralizar informações, facilitar o acompanhamento dos processos internos e garantir maior controle e segurança nas operações da empresa.

PROBLEMA E JUSTIFICATIVA

O Pet Shop Pata Feliz busca solucionar problemas relacionados à organização e ao controle das informações da empresa. Entre as principais dificuldades estão o gerenciamento de cadastros de clientes e animais, a organização dos agendamentos e o acompanhamento dos serviços realizados. A falta de integração dessas informações pode causar registros duplicados, conflitos de horários e dificuldades no atendimento.

Na parte administrativa, os principais desafios envolvem o controle de estoque, o acompanhamento das compras e vendas e a gestão financeira. A ausência de um sistema centralizado pode dificultar a identificação de produtos disponíveis, o controle de pagamentos, o registro de despesas e a conferência das movimentações financeiras.

Para solucionar esses problemas, o sistema proposto permitirá centralizar os dados em um único banco de dados, facilitando o acesso às informações e a realização das atividades diárias. Dessa forma, será possível reduzir erros, melhorar a organização dos processos, agilizar os atendimentos e proporcionar maior eficiência e segurança na administração do Pet Shop Pata Feliz.

4. OBJETIVOS DO SISTEMA

4.1 Objetivo geral

Desenvolver um banco de dados para o sistema de gestão do Pet Shop Pata Feliz, com o objetivo de organizar, armazenar e gerenciar as informações relacionadas aos clientes, animais, funcionários, produtos, serviços, agendamentos, vendas, estoque e movimentações financeiras. O sistema busca integrar os processos operacionais e administrativos da empresa, garantindo maior segurança, eficiência e confiabilidade das informações, além de facilitar o atendimento aos clientes e a tomada de decisões.

4.2 Objetivos específicos

- Identificar e definir os requisitos funcionais e não funcionais do sistema, considerando as necessidades do Pet Shop Pata Feliz.
- Estabelecer as regras de negócio para garantir a organização e o funcionamento dos processos da empresa.
- Elaborar o Diagrama Entidade-Relacionamento (DER), representando as entidades, seus atributos e relacionamentos.
- Desenvolver o modelo lógico e o dicionário de dados, definindo as tabelas, os campos e as chaves primárias e estrangeiras.
- Implementar o banco de dados utilizando a linguagem SQL, permitindo o armazenamento e o gerenciamento das informações.
- Organizar os cadastros de clientes, animais, funcionários, produtos, serviços e fornecedores.
- Facilitar o controle de agendamentos, atendimentos, vendas, compras e movimentações de estoque.
- Desenvolver consultas SQL para acompanhar as movimentações financeiras, os pagamentos e os resultados administrativos.
- Garantir a integridade, a segurança e a confiabilidade dos dados armazenados.
- Contribuir para a melhoria dos processos internos e apoiar a tomada de decisões da empresa.

5. REQUISITOS DO SISTEMA

5.1 Requisitos funcionais

Código	Descrição do requisito funcional
RF01	O sistema deverá permitir que funcionários autenticados e autorizados cadastrem clientes, animais, produtos e serviços.
RF02	O sistema deverá permitir que usuários com permissão administrativa cadastrem, consultem, alterem e desativem funcionários, definindo seus perfis de acesso.
RF03	O sistema deverá permitir que atendentes registrem, alterem e cancelem agendamentos, associando cliente, animal, serviço, data e horário.
RF04	O sistema deverá disponibilizar consulta dos agendamentos por data, horário, cliente, animal, serviço e situação.
RF05	O sistema deverá gerar relatórios de vendas com filtros por cliente e período.
RF06	O sistema deverá permitir consultar e gerar relatórios de pagamentos por período, venda ou atendimento relacionado, forma de pagamento e situação.
RF07	O sistema deverá permitir consultar o estoque por produto e fornecedor, com agrupamento por categoria caso essa classificação seja definida no cadastro.
RF08	O sistema deverá permitir consultar o histórico de atendimentos de um animal, incluindo data, serviço, funcionário responsável e observações.
RF09	O sistema deverá permitir consultar o histórico de compras de um cliente, incluindo data, produtos, quantidades, valores e situação.
RF10	O sistema deverá permitir registrar vendas vinculadas a um cliente.
RF11	O sistema deverá permitir registrar os produtos, quantidades, preços unitários e subtotais que compõem cada venda.
RF12	O sistema deverá permitir registrar a execução dos serviços, associando atendimento, animal, serviço, funcionário, data e observações.

RF13	O sistema deverá atualizar o estoque após a confirmação de vendas e permitir registrar entradas de mercadorias recebidas.
RF14	O sistema deverá permitir consultar produtos e serviços cadastrados e seus respectivos dados.

5.2 Requisitos não funcionais

Código	Descrição do requisito não funcional
RNF01	O sistema deverá autenticar os usuários e aplicar autorização baseada em perfis e permissões.
RNF02	O tratamento de dados pessoais deverá observar os requisitos aplicáveis da LGPD, limitando à coleta ao necessário e restringindo o acesso conforme a função do usuário.
RNF03	Sob a carga operacional definida, pelo menos 95% das consultas de pesquisa e recuperação deverão responder em até 2 segundos.
RNF04	A interface deverá permitir que funcionários com conhecimentos básicos de informática concluam as tarefas principais sem treinamento formal, conforme critérios de teste de usabilidade definidos.
RNF05	O sistema deverá proteger dados pessoais e informações sensíveis por criptografia durante o armazenamento e a transmissão, com gerenciamento seguro de chaves.
RNF06	O sistema deverá registrar operações relevantes dos usuários, incluindo usuário, data e hora, operação e resultado, protegendo os registros contra alterações não autorizadas.
RNF07	O sistema deverá realizar backups periódicos e permitir restauração conforme objetivos de recuperação definidos.
RNF08	O sistema deverá atender a uma meta de disponibilidade durante o horário de funcionamento, definida pelo grupo.
RNF09	O sistema deverá validar entradas e preservar a integridade dos dados por meio de regras e restrições consistentes.
RNF10	O sistema deverá disponibilizar análises gerenciais assistidas por IA com base em dados autorizados de vendas e agendamentos, sem realizar alterações transacionais automaticamente.

5.3 Regras de negócio

Código	Regra de negócio
RN01	Um cliente pode possuir vários animais.
RN02	Um animal pertence a apenas um cliente.
RN03	Um cliente pode realizar várias compras.
RN04	Uma compra pertence a apenas um cliente.
RN05	Uma compra deve possuir pelo menos um produto.
RN06	Um produto pode estar presente em várias compras.
RN07	Um produto pode possuir vários fornecedores.
RN08	Um fornecedor pode fornecer vários produtos.
RN09	Um animal pode realizar vários serviços ao longo do tempo.
RN10	Um serviço realizado deve estar associado a um animal.
RN11	Um funcionário pode realizar vários atendimentos.
RN12	Cada atendimento deve ser realizado por um funcionário.

6. MODELAGEM CONCEITUAL E DER

6.1 Entidades identificadas no DER

Entidade	Finalidade
TUTOR	Pessoa responsável pelos pets e por solicitar serviços e pedidos.
PET	Animal cadastrado no pet shop.
TUTELA	Associação entre tutor e pet.
FUNCIONARIO	Colaborador que realiza atendimentos.
AGENDAMENTO	Reserva de data e horário para atendimento.
ATENDIMENTO	Registro da execução de serviço para um pet.
SERVICO	Serviço oferecido pelo pet shop.
ITEM_ATENDIMENTO	Serviço individual incluído em um atendimento.
PEDIDO	Registro de venda solicitada por um tutor.
ITEM_PEDIDO	Produto e quantidade que compõem um pedido.
PRODUTO	Mercadoria comercializada e controlada no estoque.
MOVIMENTACAO_ESTOQUE	Histórico de entradas, saídas e ajustes de estoque.
FORNECEDOR	Empresa que fornece produtos ao pet shop.
PRODUTO_FORNECEDOR	Associação entre produtos e seus fornecedores.
COMPRA	Aquisição de mercadorias de fornecedor.
ITEM_COMPRA	Produtos adquiridos em cada compra.
CONTA_A_RECEBER	Parcela ou obrigação financeira decorrente de pedido.
PAGAMENTO	Recebimento realizado pelo cliente.
TAXA_OPERADORA	Regra de cobrança de taxas por meio de pagamento.
MOVIMENTO_CAIXA	Entrada ou saída financeira registrada.
CONCILIACAO	Conferência entre movimento registrado e extrato.
DESPESA	Gasto ou obrigação operacional do negócio.
CONTA_A_PAGAR	Obrigação financeira com vencimento e possível parcelamento.
BAIXA_PAGAR	Registro de quitação de conta a pagar.

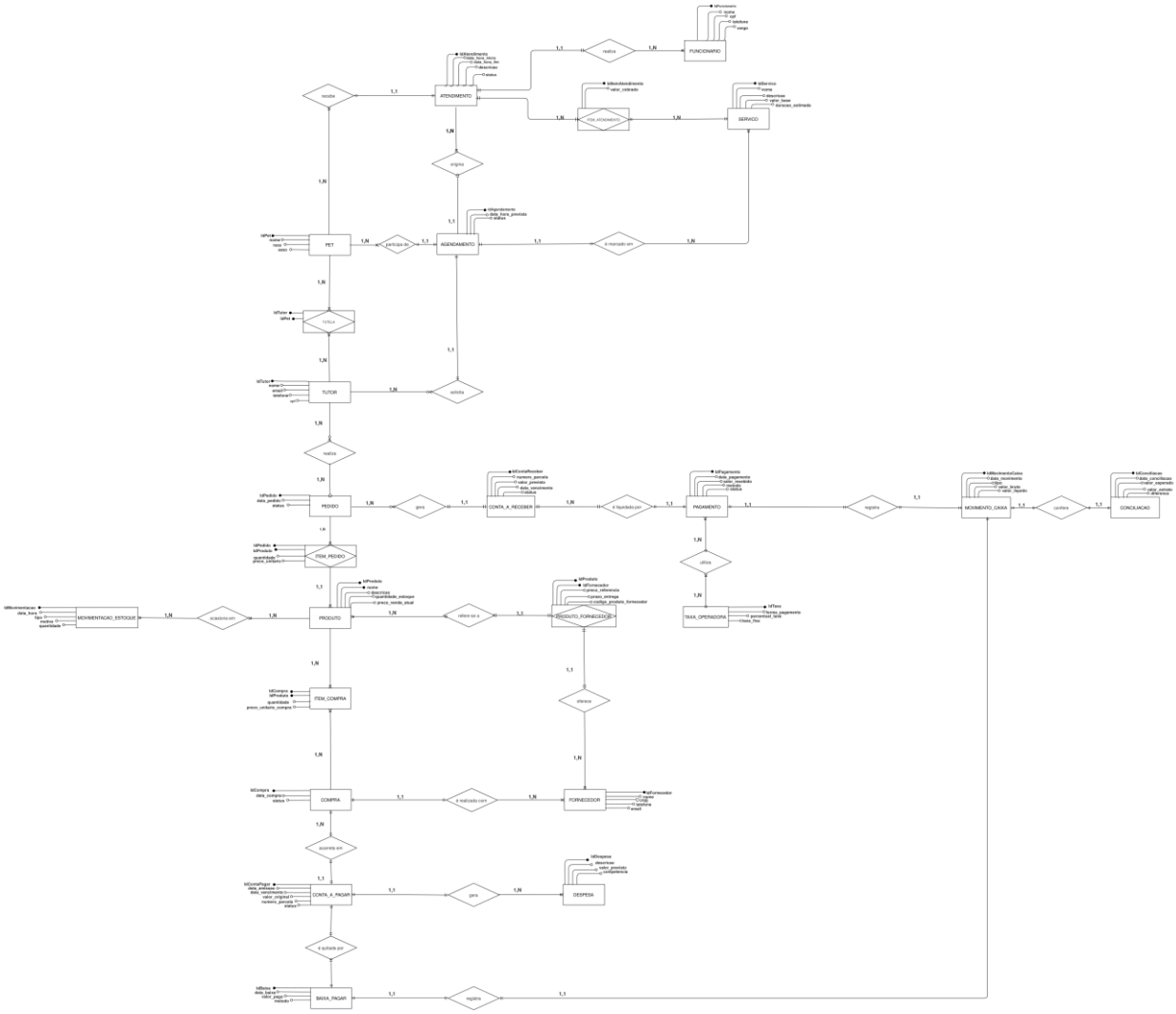
6.2 Relacionamentos representados

- TUTOR — TUTELA — PET.
- TUTOR — AGENDAMENTO; PET — AGENDAMENTO; AGENDAMENTO — ATENDIMENTO; PET — ATENDIMENTO.
- FUNCIONARIO — ATENDIMENTO.
- ATENDIMENTO — ITEM_ATENDIMENTO — SERVICO.
- TUTOR — PEDIDO; PEDIDO — ITEM_PEDIDO — PRODUTO.
- PRODUTO — MOVIMENTACAO_ESTOQUE.
- PRODUTO — PRODUTO_FORNECEDOR — FORNECEDOR.
- COMPRA — ITEM_COMPRA — PRODUTO; COMPRA — FORNECEDOR.
- PEDIDO — CONTA_A_RECEBER — PAGAMENTO — MOVIMENTO_CAIXA — CONCILIACAO.
- PAGAMENTO — TAXA_OPERADORA.
- COMPRA — CONTA_A_PAGAR — BAIXA_PAGAR; CONTA_A_PAGAR — DESPESA.

6.3 Entidades associativas

Entidade associativa	Relacionamento resolvido	Chaves propostas para o modelo lógico
TUTELA	TUTOR × PET	(IdTutor, IdPet)
ITEM_PEDIDO	PEDIDO × PRODUTO	(IdPedido, IdProduto), validar repetição do mesmo produto no pedido
PRODUTO_FORNECEDOR	PRODUTO × FORNECEDOR	(IdProduto, IdFornecedor)
ITEM_COMPRA	COMPRA × PRODUTO	(IdCompra, IdProduto), validar repetição do produto na compra
ITEM_ATENDIMENTO	ATENDIMENTO × SERVICO	IdItemAtendimento no DER; validar FKs e necessidade de chave composta

6.4 DER encaminhado



7. DICIONÁRIO DE DADOS

O dicionário encaminhado é conceitual: descreve o significado das entidades e dos atributos apresentados no DER. Ainda será necessário completar os tipos SQL, tamanhos, nulidade, PK/FK, UNIQUE, DEFAULT e demais restrições.

Entidade	Atributos registrados no dicionário
TUTOR	IdTutor; nome; email; telefone; cpf
PET	IdPet; nome; raca; sexo
TUTELA	IdTutor; IdPet
FUNCIONARIO	IdFuncionario; nome; cpf; telefone; cargo
AGENDAMENTO	IdAgendamento; data_hora_prevista; status
ATENDIMENTO	IdAtendimento; data_hora_inicio; data_hora_fim; descricao; status
SERVICO	IdServico; nome; descricao; valor_base; duracao_estimada
ITEM_ATENDIMENTO	IdItemAtendimento; valor_cobrado
PEDIDO	IdPedido; data_pedido; status
ITEM_PEDIDO	IdPedido; IdProduto; quantidade; preco_unitario
PRODUTO	IdProduto; nome; descricao; quantidade_estoque; preco_venda_atual
MOVIMENTACAO_ESTOQUE	IdMovimentacao; data_hora; tipo; motivo; quantidade
FORNECEDOR	IdFornecedor; nome; cnpj; telefone; email
PRODUTO_FORNECEDOR	IdProduto; IdFornecedor; preco_referencia; prazo_entrega; codigo_produto_fornecedor
COMPRA	IdCompra; data_compra; status
ITEM_COMPRA	IdCompra; IdProduto; quantidade; preco_unitario_compra
CONTA_A_RECEBER	IdContaReceber; numero_parcela; valor_previsto; data_vencimento; status
PAGAMENTO	IdPagamento; data_pagamento; valor_recebido; metodo; status
TAXA_OPERADORA	IdTaxa; forma_pagamento; percentual_taxa; taxa_fixa
MOVIMENTO_CAIXA	IdMovimentoCaixa; data_movimento; tipo; valor_bruto; valor_liquido
CONCILIACAO	IdConciliacao; data_conciliacao; valor_esperado; valor_extrato; diferenca
DESPESA	IdDespesa; descricao; valor_previsto; competencia
CONTA_A_PAGAR	IdContaPagar; data_emissao; data_vencimento; valor_original; numero_parcela; status
BAIXA_PAGAR	IdBaixa; data_baixa; valor_pago; metodo

8. DIÁRIOS DE BORDO E DECISÕES DE MODELAGEM

8.1 Diário de bordo 1 — Modelagem conceitual avançada

- O primeiro diário propôs TUTELA para representar a associação entre múltiplos tutores e pets.
- A revisão orienta utilizar TUTELA no DER atual e considerar (IdTutor, IdPet) como chave primária composta, com ambos os campos também como FKs.
- Na versão anterior de PET havia Id_cliente e atributos que não aparecem no DER atual. O dicionário atualizado limita PET a IdPet, nome, raca e sexo.
- Se o grupo optar pelas RN01 e RN02, que definem um único tutor por pet, será necessário rever a necessidade de TUTELA.

8.2 Diário de bordo 2 — Auditoria do modelo lógico

- TUTELA deve ser classificada como entidade associativa, não forte.
- Em FUNCIONARIO–ATENDIMENTO, FUNCIONARIO fica no lado 1 e ATENDIMENTO no lado N. A FK IdFuncionario deve ficar em ATENDIMENTO.
- Um pet pode receber vários atendimentos e cada atendimento se refere a um pet; o grupo ainda deve confirmar se a participação mínima do pet é obrigatória.
- Os exemplos e documentos devem utilizar os nomes e atributos do DER novo.

8.3 Alterações registradas

Tema	Encaminhamento atual
Nomenclatura	Padronizar TUTOR, PET e PEDIDO conforme o DER novo.
TUTELA	Validar se será mantida como associação N:N ou substituída por vínculo 1:N conforme RN01/RN02.
Fornecedor e produto	DER inclui PRODUTO_FORNECEDOR.
Itens de compra	DER inclui ITEM_COMPRA.
Atendimento e serviço	DER inclui ITEM_ATENDIMENTO; cardinalidades ainda precisam de validação.
Financeiro	DER inclui CONTA_A_RECEBER, PAGAMENTO, MOVIMENTO_CAIXA, CONCILIACAO e TAXA_OPERADORA.
Contas a pagar	DER inclui CONTA_A_PAGAR, DESPESA e BAIXA_PAGAR.

9. MODELAGEM LÓGICA

Tabela	PK proposta	FKs propostas
TUTOR	IdTutor	—
PET	IdPet	— ou vínculo via TUTELA
TUTELA	IdTutor + IdPet	IdTutor → TUTOR; IdPet → PET
FUNCIONARIO	IdFuncionario	—
AGENDAMENTO	IdAgendamento	IdTutor → TUTOR; IdPet → PET (validar)
ATENDIMENTO	IdAtendimento	IdPet → PET; IdAgendamento → AGENDAMENTO; IdFuncionario → FUNCIONARIO
SERVICO	IdServico	—
ITEM_ATENDIMENTO	IdItemAtendimento	IdAtendimento → ATENDIMENTO; IdServico → SERVICO
PEDIDO	IdPedido	IdTutor → TUTOR
ITEM_PEDIDO	IdPedido + IdProduto	IdPedido → PEDIDO; IdProduto → PRODUTO
PRODUTO	IdProduto	—
MOVIMENTACAO_ESTOQUE	IdMovimentacao	IdProduto → PRODUTO
FORNECEDOR	IdFornecedor	—
PRODUTO_FORNECEDOR	IdProduto + IdFornecedor	IdProduto → PRODUTO; IdFornecedor → FORNECEDOR
COMPRA	IdCompra	IdFornecedor → FORNECEDOR
ITEM_COMPRA	IdCompra + IdProduto	IdCompra → COMPRA; IdProduto → PRODUTO
CONTA_A_RECEBER	IdContaReceber	IdPedido → PEDIDO
PAGAMENTO	IdPagamento	IdContaReceber → CONTA_A_RECEBER; IdTaxa → TAXA_OPERADORA (validar)
TAXA_OPERADORA	IdTaxa	—
MOVIMENTO_CAIXA	IdMovimentoCaixa	IdPagamento → PAGAMENTO (modelo atual evidencia recebimentos)
CONCILIACAO	IdConciliacao	IdMovimentoCaixa → MOVIMENTO_CAIXA (validar)
DESPESA	IdDespesa	IdContaPagar → CONTA_A_PAGAR (validar direção)
CONTA_A_PAGAR	IdContaPagar	IdCompra → COMPRA (validar outras origens)
BAIXA_PAGAR	IdBaixa	IdContaPagar → CONTA_A_PAGAR; vínculo a MOVIMENTO_CAIXA a confirmar

10. NORMALIZAÇÃO

10.1 Primeira Forma Normal (1FN)

A Primeira Forma Normal (1FN) estabelece que cada atributo de uma tabela deve conter apenas um valor, evitando informações repetidas ou múltiplos dados armazenados em um mesmo campo. Essa organização facilita o armazenamento, a consulta e a atualização das informações no banco de dados.

No sistema do Pet Shop Pata Feliz, as tabelas foram estruturadas para que cada campo armazene uma informação específica. Por exemplo, na tabela PET, os atributos como nome, espécie e data de nascimento são registrados separadamente. Da mesma forma, na tabela PRODUTO, informações como nome, preço e quantidade em estoque são armazenadas em campos individuais.

Além disso, os relacionamentos que podem envolver vários registros foram organizados por meio de tabelas específicas, como ITEM_PEDIDO, que permite registrar os produtos de cada pedido sem criar grupos repetidos de atributos. Dessa forma, a estrutura proposta atende aos princípios da Primeira Forma Normal (1FN), contribuindo para a organização e a integridade dos dados.

10.2 Segunda Forma Normal (2FN)

A Segunda Forma Normal (2FN) estabelece que uma tabela deve estar na Primeira Forma Normal (1FN) e que todos os atributos não-chave devem depender completamente da chave primária, evitando dependências parciais e a repetição desnecessária de informações.

No banco de dados do Pet Shop Pata Feliz, as tabelas foram organizadas para que os atributos estejam relacionados às suas respectivas chaves primárias. Por exemplo, na tabela PRODUTO, informações como nome e preço dependem diretamente do identificador do produto. Já na tabela ITEM_PEDIDO, os dados referentes aos itens de uma venda são armazenados separadamente das informações gerais do pedido e dos produtos.

Dessa forma, a estrutura proposta busca atender aos princípios da Segunda Forma Normal (2FN), garantindo que os atributos não-chave dependam integralmente de suas respectivas chaves primárias. Essa organização contribui para reduzir a duplicidade de dados, evitar inconsistências e facilitar a manutenção das informações no sistema.

10.3 Terceira Forma Normal (3FN)

A Terceira Forma Normal (3FN) estabelece que uma tabela deve estar na Segunda Forma Normal (2FN) e que todos os atributos não-chave devem depender diretamente da chave primária, evitando dependências transitivas. Isso significa que um atributo não-chave não deve depender de outro atributo não-chave.

No banco de dados do Pet Shop Pata Feliz, as informações foram organizadas em tabelas específicas, de acordo com suas funções. Por exemplo, na tabela PEDIDO, os dados da venda são armazenados separadamente dos dados do cliente, que ficam na tabela TUTOR. Da mesma forma, as informações dos fornecedores são registradas na tabela FORNECEDOR, enquanto os produtos são armazenados na tabela PRODUTO, utilizando chaves estrangeiras e tabelas de relacionamento para estabelecer as associações necessárias.

Dessa forma, a estrutura proposta busca atender aos princípios da Terceira Forma Normal (3FN), reduzindo a redundância de informações e evitando problemas de inserção, atualização e exclusão de dados. Essa organização contribui para garantir maior integridade, consistência e facilidade na manutenção do banco de dados.

11. IMPLEMENTAÇÃO SQL

O Script 01 (01_schema.sql) cria o banco pata_feliz e 24 tabelas no MySQL 8.0+, utilizando InnoDB, utf8mb4, chaves primárias e estrangeiras, índices, ENUM e restrições CHECK. Abrange cadastros, agendamentos, atendimentos, produtos, estoque, fornecedores, pedidos, compras, contas a receber, contas a pagar, caixa e conciliação. O código integral foi preservado no Apêndice A.

12. DADOS PARA TESTES

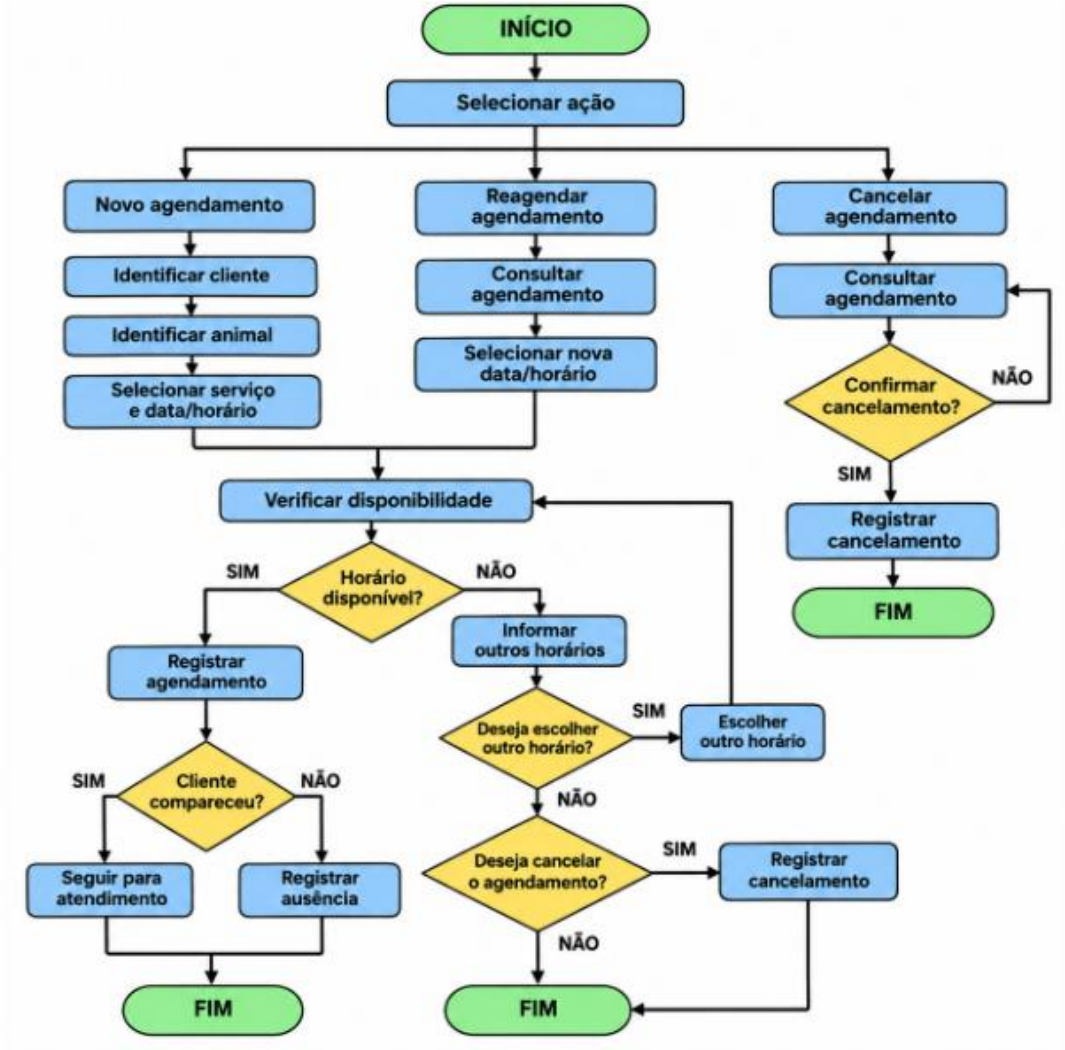
O Script 02 (02_inserção_dados.sql) insere dados fictícios para testar cadastros, pets, tutores, funcionários, agendamentos, atendimentos, vendas, estoque, compras, pagamentos, despesas, baixas, movimentos de caixa e conciliações. Utiliza transação e consulta final de contagem de registros. O código integral foi preservado no Apêndice B.

13. CONSULTAS GERENCIAIS E DE AUDITORIA

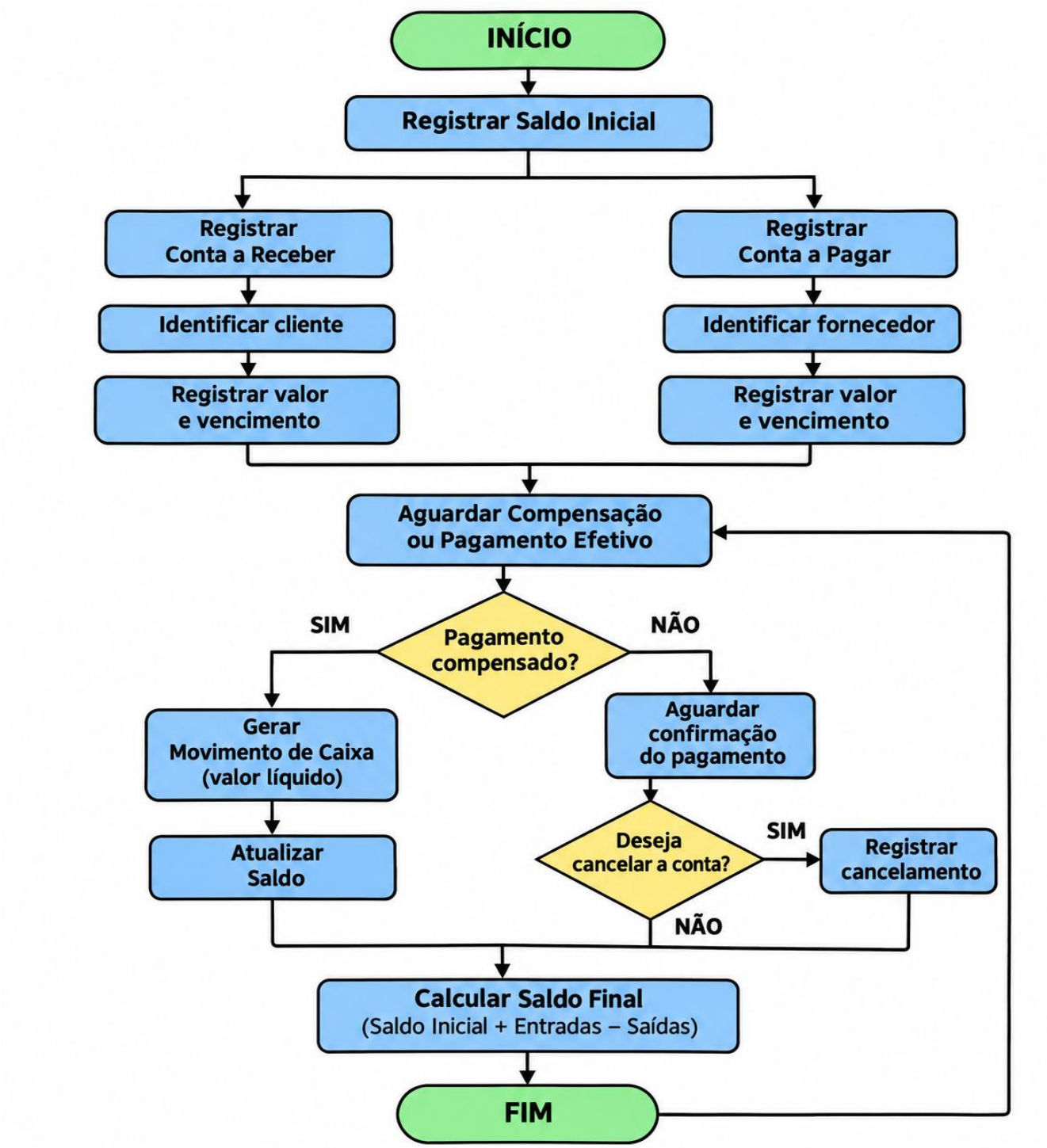
O Script 03 (03_consultas.sql) reúne 28 consultas SQL organizadas em oito grupos: (A) cadastros, tutores e pets; (B) agendamentos e atendimentos; (C) produtos, estoque e fornecedores; (D) pedidos e compras; (E) contas a receber e pagamentos; (F) contas a pagar e despesas; (G) caixa e conciliação; (H) auditoria e integridade operacional. O código integral foi preservado no Apêndice C.

14. FLUXOGRAMAS

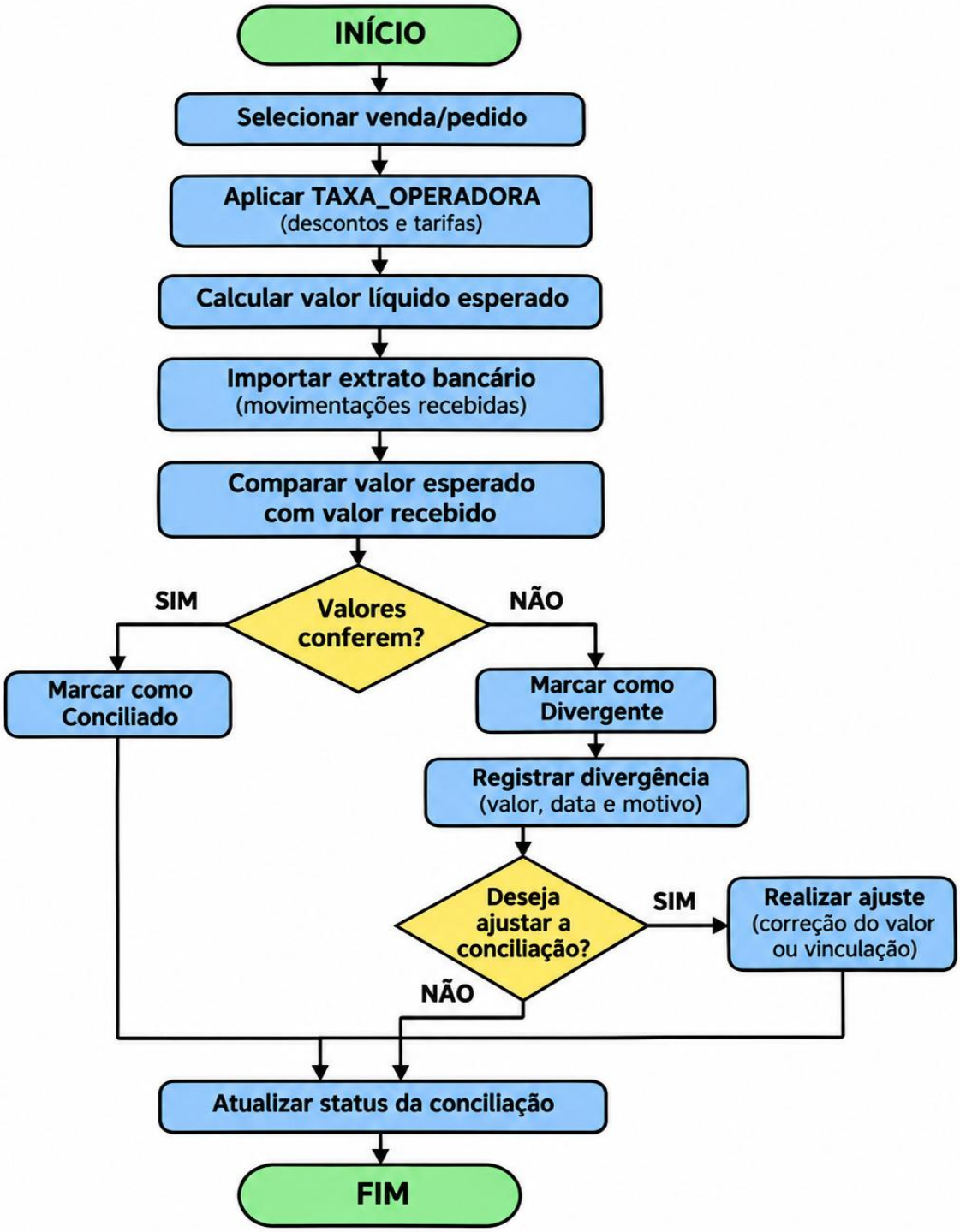
14.1 Agendamento, reagendamento e cancelamento



14.2 Fluxo de caixa



14.3 Conciliação financeira



15. CONTROLE FINANCEIRO E CONCILIAÇÃO

A proposta financeira separa obrigações e valores previstos dos movimentos efetivos de dinheiro. CONTA_A_RECEBER representa o direito de receber; CONTA_A_PAGAR representa a obrigação de pagar; PAGAMENTO e BAIXA_PAGAR registram liquidações; MOVIMENTO_CAIXA deve refletir entradas e saídas efetivas.

15.1 MOVIMENTO_CAIXA

Atributo proposto	Descrição
IdMovimentoCaixa (PK)	Identificador do movimento.
data_movimento	Data em que o dinheiro entrou ou saiu de fato.
tipo	Natureza: ENTRADA ou SAIDA.
valor_bruto	Valor antes dos descontos.
valor_liquido	Valor que efetivamente afeta o saldo.

A proposta anterior do grupo também mencionou categoria, valor_taxa, id_origem e id_conciliacao. Confirmar a inclusão desses campos no DER, dicionário e modelo lógico.

Saldo Final = Saldo Inicial + soma das entradas líquidas − soma das saídas líquidas. O saldo inicial de cada dia corresponde ao saldo final do dia anterior.

Exemplo: saldo inicial R$ 1.000,00; venda a crédito de R$ 100,00 ainda não compensada não entra no caixa do dia; pagamento efetivo ao fornecedor de R$ 200,00 gera saída; saldo final = R$ 800,00.

15.2 CONTA_A_RECEBER e PAGAMENTO

CONTA_A_RECEBER registra valor previsto, parcela, vencimento e status. PAGAMENTO registra a data e o valor efetivamente recebido, método e status. O script encaminhado já cria essas duas tabelas e liga PAGAMENTO a CONTA_A_RECEBER.

15.3 CONTA_A_PAGAR e BAIXA_PAGAR

CONTA_A_PAGAR registra emissão, vencimento, valor original, parcela e status. BAIXA_PAGAR registra a quitação, com data, valor pago e método. O script encaminhado já cria essas tabelas e estabelece a FK da baixa para a conta.

15.4 TAXA_OPERADORA e CONCILIACAO

TAXA_OPERADORA registra forma de pagamento, percentual e taxa fixa. CONCILIACAO compara valor esperado com valor do extrato e registra diferença. O DER contém data_conciliacao, valor_esperado, valor_extrato e diferenca.

Regra proposta RN16: se ABS(diferenca) > R$ 0,05, marcar Divergente e gerar alerta. Causas previstas: taxa maior que a contratada, chargeback/estorno e venda cancelada pela operadora.

15.5 Pendências financeiras

- Confirmar se conta a receber pode ter múltiplos pagamentos parciais.
- Confirmar se conta a pagar pode ter múltiplas baixas parciais.
- Confirmar ligação de MOVIMENTO_CAIXA com recebimentos e também com BAIXA_PAGAR.
- Definir como serão registrados estornos e chargebacks.
- Definir a cardinalidade entre CONCILIACAO e MOVIMENTO_CAIXA.
- Definir vínculo de DESPESA com CONTA_A_PAGAR e permitir despesas que não tenham origem em COMPRA, se necessário.
- Completar TAXA_OPERADORA no SQL e ligar a forma de pagamento utilizada.

16. INOVAÇÃO E VALOR AGREGADO

16.1 Ficha de Cuidados e Retorno Programado

A proposta apresentada anteriormente é transformar o histórico dos pets em acompanhamento, permitindo identificar animais com retorno recomendado e facilitar o contato com seus tutores.

16.2 Funcionamento proposto

- Registrar observações sobre o pet durante o atendimento, como temperamento, restrições e preferências.
- Disponibilizar essas observações em atendimentos futuros.
- Definir um intervalo de retorno recomendado para cada serviço.
- Gerar uma lista de pets com retorno próximo ou vencido a partir do último atendimento.
- Permitir que a equipe entre em contato com o tutor.

16.3 Benefícios e indicadores

- Atendimento mais personalizado e continuidade do cuidado.
- Melhor organização dos retornos e incentivo à recorrência dos clientes.
- Indicadores: percentual de atendimentos com observações; retornos realizados no prazo; agendamentos gerados após contato.

A proposta utiliza informações de PET, TUTOR, ATENDIMENTO e SERVICO. O grupo deve validar se adicionará ao SERVICO um campo de intervalo de retorno recomendado.

16.4 Justificativa de MOVIMENTO_CAIXA

A entidade MOVIMENTO_CAIXA foi incluída para diferenciar pedidos registrados de dinheiro efetivamente disponível, permitindo calcular saldo inicial e saldo final diário, requisito importante de um sistema de gestão.

17. BI E INTELIGÊNCIA ARTIFICIAL

A utilização de Business Intelligence (BI) e Inteligência Artificial (IA) poderá contribuir para melhorar a gestão do Pet Shop Pata Feliz. Por meio dos dados armazenados, será possível gerar relatórios e indicadores sobre vendas, estoque, serviços realizados e movimentações financeiras, auxiliando na tomada de decisões.

Além disso, a Inteligência Artificial poderá analisar o histórico de compras e atendimentos, prever a demanda por produtos e enviar lembretes de cuidados e retornos programados para os animais. Dessa forma, essas tecnologias poderão melhorar a organização, reduzir custos e aumentar a eficiência da empresa.

18. CONCLUSÃO

O desenvolvimento deste projeto permitiu compreender a importância de um banco de dados bem estruturado para a organização e o gerenciamento das informações do Pet Shop Pata Feliz. Por meio da definição das regras de negócio, dos requisitos do sistema e da modelagem de dados, foi possível propor uma solução para melhorar os processos operacionais e administrativos da empresa.

Durante o trabalho, foram aplicados conhecimentos relacionados ao Diagrama Entidade-Relacionamento (DER), à normalização de dados e à linguagem SQL, permitindo estruturar o armazenamento e a consulta de informações sobre clientes, animais, produtos, serviços, agendamentos e movimentações financeiras.

Conclui-se que o projeto contribuiu para o desenvolvimento dos conhecimentos adquiridos ao longo da disciplina, demonstrando como a utilização de um banco de dados pode facilitar o controle das atividades, reduzir erros e apoiar a tomada de decisões, proporcionando maior organização e eficiência à empresa.

19. REFERÊNCIAS E ANEXOS

19.1 Repositório do projeto no GitHub

O projeto acadêmico Pata Feliz possui um repositório público no GitHub para organizar e compartilhar seus arquivos, documentação e atualizações.

Repositório: https://github.com/criszst/PetShop-PataFeliz

- Anexo A — DER_PataFeliz.pdf.
- Anexo B — Dicionário de dados e revisão dos diários de bordo.
- Anexo C — Script sistema_financeiro.sql.
- Anexo D — Fluxograma de agendamento.
- Anexo E — Scripts SQL completos e evidências de execução.

20. CHECKLIST FINAL DO GRUPO

- ☐ Resolver a divergência entre RN01/RN02 e TUTELA no DER.
- ☐ Conferir todas as cardinalidades mínimas e máximas.
- ☐ Validar AGENDAMENTO–ATENDIMENTO, inclusive cancelamentos e atendimentos sem agendamento.
- ☐ Validar ITEM_ATENDIMENTO e a relação entre atendimento e serviço.
- ☐ Validar se produto pode repetir no mesmo pedido/compra antes de definir PK composta.
- ☐ Completar tipos, tamanhos, nulidade, unicidade e restrições do dicionário.
- ☐ Corrigir o erro de nome da PK em contas_a_pagar no SQL.
- ☐ Completar o SQL de todas as entidades e validar as FKs.
- ☐ Definir pagamentos e baixas parciais, taxas, estornos e conciliação.
- ☐ Completar normalização, massa de dados, consultas gerenciais e auditoria.
- ☐ Acrescentar fluxogramas visuais financeiros, BI/IA, conclusão e referências.
- ☐ Conferir consistência entre DER, regras, dicionário, modelo lógico e scripts SQL.

APÊNDICES — SCRIPTS SQL INTEGRAIS

Os scripts a seguir são transcrições integrais dos arquivos enviados pelo grupo. Para executar: 01_schema.sql → 02_inserção_dados.sql → 03_consultas.sql. Não executar novamente o primeiro script após inserir dados, pois contém DROP DATABASE IF EXISTS.

APÊNDICE A — CRIAÇÃO DO BANCO E DAS TABELAS

Arquivo original: 01_schema(1).sql | SGBD: MySQL 8.0+

```
/* ====================================================================
   PATA FELIZ - Sistema de Gestão
   Script 01 - Criação do banco e das tabelas (DDL)
   SGBD: MySQL 8.0+ (testado na 8.0.46)  |  Engine: InnoDB  |  utf8mb4

   Base de referência: DER_PataFeliz.pdf (nomenclatura TUTOR/PET/PEDIDO).
   Convenções:
     - tabelas e colunas em snake_case, nome da tabela = nome da entidade
       do DER em minúsculas (IdTutor -> id_tutor);
     - PK substituta AUTO_INCREMENT nas entidades fortes; PK composta nas
       entidades associativas (tutela, item_pedido, item_compra,
       produto_fornecedor);
     - valores monetários DECIMAL(12,2); percentuais DECIMAL(5,2);
     - status como ENUM (valores em MAIÚSCULAS, sem acento).
   Comentários marcados com [DECISÃO] indicam pontos em que o DER não
   define a regra e foi adotada a interpretação descrita.
   ==================================================================== */

DROP DATABASE IF EXISTS pata_feliz;
CREATE DATABASE pata_feliz
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
USE pata_feliz;

/* --------------------------------------------------------------------
   1. CADASTROS BÁSICOS
   -------------------------------------------------------------------- */

CREATE TABLE tutor (
  id_tutor   INT UNSIGNED NOT NULL AUTO_INCREMENT,
  nome       VARCHAR(100) NOT NULL,
  email      VARCHAR(120) NULL,
  telefone   VARCHAR(20)  NOT NULL,
  cpf        CHAR(11)     NULL,             -- só dígitos; opcional (LGPD: coleta mínima)
  PRIMARY KEY (id_tutor),
  UNIQUE KEY uq_tutor_email (email),
  UNIQUE KEY uq_tutor_cpf (cpf),
  CONSTRAINT ck_tutor_cpf CHECK (cpf IS NULL OR cpf REGEXP '^[0-9]{11}$')
) ENGINE=InnoDB COMMENT='Responsável pelos pets (solicita serviços e realiza pedidos)';

CREATE TABLE pet (
  id_pet  INT UNSIGNED NOT NULL AUTO_INCREMENT,
  nome    VARCHAR(80)  NOT NULL,
  raca    VARCHAR(60)  NULL,
  sexo    CHAR(1)      NULL,                 -- M = macho, F = fêmea
  PRIMARY KEY (id_pet),
  CONSTRAINT ck_pet_sexo CHECK (sexo IS NULL OR sexo IN ('M','F'))
) ENGINE=InnoDB COMMENT='Animal cadastrado; vínculo com o tutor via TUTELA';

/* TUTELA: associação N:N entre TUTOR e PET (conforme o DER).
   [DECISÃO] Mantida como no DER. Conflita com a RN02 (animal pertence a um
   único cliente) - ver relatório final. */
CREATE TABLE tutela (
  id_tutor  INT UNSIGNED NOT NULL,
  id_pet    INT UNSIGNED NOT NULL,
  PRIMARY KEY (id_tutor, id_pet),
  KEY ix_tutela_pet (id_pet),
  CONSTRAINT fk_tutela_tutor FOREIGN KEY (id_tutor) REFERENCES tutor (id_tutor)
    ON DELETE CASCADE,
  CONSTRAINT fk_tutela_pet FOREIGN KEY (id_pet) REFERENCES pet (id_pet)
    ON DELETE CASCADE
) ENGINE=InnoDB COMMENT='Entidade associativa TUTOR x PET';

CREATE TABLE funcionario (
  id_funcionario  INT UNSIGNED NOT NULL AUTO_INCREMENT,
  nome            VARCHAR(100) NOT NULL,
  cpf             CHAR(11)     NOT NULL,
  telefone        VARCHAR(20)  NULL,
  cargo           VARCHAR(50)  NOT NULL,
  PRIMARY KEY (id_funcionario),
  UNIQUE KEY uq_funcionario_cpf (cpf),
  CONSTRAINT ck_funcionario_cpf CHECK (cpf REGEXP '^[0-9]{11}$')
) ENGINE=InnoDB COMMENT='Colaborador que realiza atendimentos';

CREATE TABLE servico (
  id_servico        INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  nome              VARCHAR(80)   NOT NULL,
  descricao         VARCHAR(255)  NULL,
```

```
  valor_base        DECIMAL(12,2) NOT NULL,
  duracao_estimada  SMALLINT UNSIGNED NOT NULL,   -- em minutos
  PRIMARY KEY (id_servico),
  UNIQUE KEY uq_servico_nome (nome),
  CONSTRAINT ck_servico_valor CHECK (valor_base >= 0),
  CONSTRAINT ck_servico_duracao CHECK (duracao_estimada > 0)
) ENGINE=InnoDB COMMENT='Serviço oferecido (banho, tosa, consulta...)';

/* ------------------------------------------------------------------
   2. AGENDAMENTO E ATENDIMENTO
   ------------------------------------------------------------------ */

/* AGENDAMENTO: no DER, um agendamento tem 1 tutor (solicita), 1 pet
   (participa de) e 1 serviço (é marcado em).
   Status derivados do fluxograma: AGENDADO -> REAGENDADO / CANCELADO /
   AUSENTE / CONCLUIDO. */
CREATE TABLE agendamento (
  id_agendamento      INT UNSIGNED NOT NULL AUTO_INCREMENT,
  data_hora_prevista  DATETIME     NOT NULL,
  status              ENUM('AGENDADO','REAGENDADO','CANCELADO','AUSENTE','CONCLUIDO')
                      NOT NULL DEFAULT 'AGENDADO',
  id_tutor            INT UNSIGNED NOT NULL,
  id_pet              INT UNSIGNED NOT NULL,
  id_servico          INT UNSIGNED NOT NULL,
  PRIMARY KEY (id_agendamento),
  KEY ix_agendamento_data (data_hora_prevista),
  KEY ix_agendamento_tutor (id_tutor),
  KEY ix_agendamento_pet (id_pet),
  KEY ix_agendamento_servico (id_servico),
  CONSTRAINT fk_agendamento_tutor   FOREIGN KEY (id_tutor)   REFERENCES tutor (id_tutor),
  CONSTRAINT fk_agendamento_pet     FOREIGN KEY (id_pet)     REFERENCES pet (id_pet),
  CONSTRAINT fk_agendamento_servico FOREIGN KEY (id_servico) REFERENCES servico (id_servico)
) ENGINE=InnoDB COMMENT='Reserva de data/hora para um serviço de um pet';

/* ATENDIMENTO: execução do serviço. RN10: associado a um pet; RN12: realizado
   por exatamente um funcionário.
   [DECISÃO] id_agendamento é opcional (atendimento sem agendamento) e UNIQUE
   (um agendamento origina no máximo um atendimento; cancelado/ausente = nenhum). */
CREATE TABLE atendimento (
  id_atendimento    INT UNSIGNED NOT NULL AUTO_INCREMENT,
  data_hora_inicio  DATETIME     NOT NULL,
  data_hora_fim     DATETIME     NULL,
  descricao         TEXT         NULL,
  status            ENUM('EM_ANDAMENTO','CONCLUIDO','CANCELADO')
                    NOT NULL DEFAULT 'EM_ANDAMENTO',
  id_pet            INT UNSIGNED NOT NULL,
  id_funcionario    INT UNSIGNED NOT NULL,
  id_agendamento    INT UNSIGNED NULL,
  PRIMARY KEY (id_atendimento),
  UNIQUE KEY uq_atendimento_agendamento (id_agendamento),
  KEY ix_atendimento_pet (id_pet, data_hora_inicio),
  KEY ix_atendimento_funcionario (id_funcionario),
  CONSTRAINT fk_atendimento_pet FOREIGN KEY (id_pet) REFERENCES pet (id_pet),
  CONSTRAINT fk_atendimento_funcionario FOREIGN KEY (id_funcionario)
    REFERENCES funcionario (id_funcionario),
  CONSTRAINT fk_atendimento_agendamento FOREIGN KEY (id_agendamento)
    REFERENCES agendamento (id_agendamento),
  CONSTRAINT ck_atendimento_periodo
    CHECK (data_hora_fim IS NULL OR data_hora_fim >= data_hora_inicio)
) ENGINE=InnoDB COMMENT='Execução de serviço(s) para um pet';

/* ITEM_ATENDIMENTO: PK substituta, como no DER (IdItemAtendimento). */
CREATE TABLE item_atendimento (
  id_item_atendimento  INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  valor_cobrado        DECIMAL(12,2) NOT NULL,
  id_atendimento       INT UNSIGNED  NOT NULL,
  id_servico           INT UNSIGNED  NOT NULL,
  PRIMARY KEY (id_item_atendimento),
  KEY ix_item_atendimento_atendimento (id_atendimento),
  KEY ix_item_atendimento_servico (id_servico),
  CONSTRAINT fk_item_atendimento_atendimento FOREIGN KEY (id_atendimento)
    REFERENCES atendimento (id_atendimento) ON DELETE CASCADE,
  CONSTRAINT fk_item_atendimento_servico FOREIGN KEY (id_servico)
    REFERENCES servico (id_servico),
  CONSTRAINT ck_item_atendimento_valor CHECK (valor_cobrado >= 0)
) ENGINE=InnoDB COMMENT='Serviço individual dentro de um atendimento';

/* ------------------------------------------------------------------
   3. PRODUTOS, ESTOQUE E FORNECEDORES
   ------------------------------------------------------------------ */

CREATE TABLE produto (
```

```
  id_produto          INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  nome                VARCHAR(100)  NOT NULL,
  descricao           VARCHAR(255)  NULL,
  quantidade_estoque  INT           NOT NULL DEFAULT 0,
  preco_venda_atual   DECIMAL(12,2) NOT NULL,
  PRIMARY KEY (id_produto),
  KEY ix_produto_nome (nome),
  CONSTRAINT ck_produto_estoque CHECK (quantidade_estoque >= 0),
  CONSTRAINT ck_produto_preco CHECK (preco_venda_atual >= 0)
) ENGINE=InnoDB COMMENT='Mercadoria vendida e controlada em estoque';

/* [DECISÃO] quantidade > 0 e o sentido vem de tipo:
   ENTRADA / AJUSTE_POSITIVO somam; SAIDA / AJUSTE_NEGATIVO subtraem. */
CREATE TABLE movimentacao_estoque (
  id_movimentacao  INT UNSIGNED NOT NULL AUTO_INCREMENT,
  data_hora        DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  tipo             ENUM('ENTRADA','SAIDA','AJUSTE_POSITIVO','AJUSTE_NEGATIVO') NOT NULL,
  motivo           VARCHAR(150) NULL,
  quantidade       INT          NOT NULL,
  id_produto       INT UNSIGNED NOT NULL,
  PRIMARY KEY (id_movimentacao),
  KEY ix_movest_produto (id_produto, data_hora),
  CONSTRAINT fk_movest_produto FOREIGN KEY (id_produto) REFERENCES produto (id_produto),
  CONSTRAINT ck_movest_quantidade CHECK (quantidade > 0)
) ENGINE=InnoDB COMMENT='Histórico de entradas, saídas e ajustes de estoque';

CREATE TABLE fornecedor (
  id_fornecedor  INT UNSIGNED NOT NULL AUTO_INCREMENT,
  nome           VARCHAR(100) NOT NULL,
  cnpj           CHAR(14)     NOT NULL,
  telefone       VARCHAR(20)  NULL,
  email          VARCHAR(120) NULL,
  PRIMARY KEY (id_fornecedor),
  UNIQUE KEY uq_fornecedor_cnpj (cnpj),
  CONSTRAINT ck_fornecedor_cnpj CHECK (cnpj REGEXP '^[0-9]{14}$')
) ENGINE=InnoDB COMMENT='Empresa que fornece produtos (RN07/RN08)';

/* PRODUTO_FORNECEDOR: N:N entre PRODUTO e FORNECEDOR (RN07/RN08). */
CREATE TABLE produto_fornecedor (
  id_produto                 INT UNSIGNED  NOT NULL,
  id_fornecedor              INT UNSIGNED  NOT NULL,
  preco_referencia           DECIMAL(12,2) NOT NULL,
  prazo_entrega              SMALLINT UNSIGNED NULL,       -- em dias
  codigo_produto_fornecedor  VARCHAR(50)   NULL,
  PRIMARY KEY (id_produto, id_fornecedor),
  KEY ix_prodforn_fornecedor (id_fornecedor),
  CONSTRAINT fk_prodforn_produto FOREIGN KEY (id_produto) REFERENCES produto (id_produto)
    ON DELETE CASCADE,
  CONSTRAINT fk_prodforn_fornecedor FOREIGN KEY (id_fornecedor)
    REFERENCES fornecedor (id_fornecedor) ON DELETE CASCADE,
  CONSTRAINT ck_prodforn_preco CHECK (preco_referencia >= 0)
) ENGINE=InnoDB COMMENT='Entidade associativa PRODUTO x FORNECEDOR';

/* ----------------------------------------------------------------------
   4. VENDAS (PEDIDO) E COMPRAS DE FORNECEDOR (COMPRA)
   ---------------------------------------------------------------------- */

CREATE TABLE pedido (
  id_pedido    INT UNSIGNED NOT NULL AUTO_INCREMENT,
  data_pedido  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  status       ENUM('ABERTO','CONFIRMADO','CANCELADO') NOT NULL DEFAULT 'ABERTO',
  id_tutor     INT UNSIGNED NOT NULL,                     -- RN04
  PRIMARY KEY (id_pedido),
  KEY ix_pedido_tutor (id_tutor, data_pedido),
  KEY ix_pedido_data (data_pedido),
  CONSTRAINT fk_pedido_tutor FOREIGN KEY (id_tutor) REFERENCES tutor (id_tutor)
) ENGINE=InnoDB COMMENT='Venda de produtos a um tutor (RN03/RN04)';

/* ITEM_PEDIDO: PK composta (id_pedido, id_produto): o mesmo produto não se
   repete no pedido (mudar a quantidade). RN05 (pedido com >= 1 item) não é
   imposta por DDL: ver trigger/procedimento no script de regras. */
CREATE TABLE item_pedido (
  id_pedido       INT UNSIGNED  NOT NULL,
  id_produto      INT UNSIGNED  NOT NULL,
  quantidade      INT           NOT NULL,
  preco_unitario  DECIMAL(12,2) NOT NULL,        -- preço praticado no momento da venda
  PRIMARY KEY (id_pedido, id_produto),
  KEY ix_item_pedido_produto (id_produto),
  CONSTRAINT fk_item_pedido_pedido FOREIGN KEY (id_pedido) REFERENCES pedido (id_pedido)
    ON DELETE CASCADE,
  CONSTRAINT fk_item_pedido_produto FOREIGN KEY (id_produto) REFERENCES produto (id_produto),
  CONSTRAINT ck_item_pedido_qtd CHECK (quantidade > 0),
```

```
  CONSTRAINT ck_item_pedido_preco CHECK (preco_unitario >= 0)
) ENGINE=InnoDB COMMENT='Entidade associativa PEDIDO x PRODUTO (subtotal = quantidade * preco_unitario)';

CREATE TABLE compra (
  id_compra      INT UNSIGNED NOT NULL AUTO_INCREMENT,
  data_compra    DATE         NOT NULL,
  status         ENUM('ABERTA','RECEBIDA','CANCELADA') NOT NULL DEFAULT 'ABERTA',
  id_fornecedor  INT UNSIGNED NOT NULL,
  PRIMARY KEY (id_compra),
  KEY ix_compra_fornecedor (id_fornecedor, data_compra),
  CONSTRAINT fk_compra_fornecedor FOREIGN KEY (id_fornecedor)
    REFERENCES fornecedor (id_fornecedor)
) ENGINE=InnoDB COMMENT='Aquisição de mercadorias junto a um fornecedor';

CREATE TABLE item_compra (
  id_compra              INT UNSIGNED  NOT NULL,
  id_produto             INT UNSIGNED  NOT NULL,
  quantidade             INT           NOT NULL,
  preco_unitario_compra  DECIMAL(12,2) NOT NULL,
  PRIMARY KEY (id_compra, id_produto),
  KEY ix_item_compra_produto (id_produto),
  CONSTRAINT fk_item_compra_compra FOREIGN KEY (id_compra) REFERENCES compra (id_compra)
    ON DELETE CASCADE,
  CONSTRAINT fk_item_compra_produto FOREIGN KEY (id_produto) REFERENCES produto (id_produto),
  CONSTRAINT ck_item_compra_qtd CHECK (quantidade > 0),
  CONSTRAINT ck_item_compra_preco CHECK (preco_unitario_compra >= 0)
) ENGINE=InnoDB COMMENT='Entidade associativa COMPRA x PRODUTO';

/* ----------------------------------------------------------------------
   5. FINANCEIRO - RECEBIMENTOS
   (tabelas do script original, corrigidas e padronizadas)
   ---------------------------------------------------------------------- */

CREATE TABLE taxa_operadora (
  id_taxa           INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  forma_pagamento   VARCHAR(50)   NOT NULL,       -- ex.: 'Visa Crédito', 'Pix'
  percentual_taxa   DECIMAL(5,2)  NOT NULL DEFAULT 0.00,
  taxa_fixa         DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  PRIMARY KEY (id_taxa),
  UNIQUE KEY uq_taxa_forma (forma_pagamento),
  CONSTRAINT ck_taxa_percentual CHECK (percentual_taxa BETWEEN 0 AND 100),
  CONSTRAINT ck_taxa_fixa CHECK (taxa_fixa >= 0)
) ENGINE=InnoDB COMMENT='Taxas cobradas por meio de pagamento';

/* CONTA_A_RECEBER: FK para PEDIDO adicionada (DER e modelo lógico).
   Uma conta por parcela do pedido. */
CREATE TABLE conta_a_receber (
  id_conta_receber  INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  numero_parcela    TINYINT UNSIGNED NOT NULL DEFAULT 1,
  valor_previsto    DECIMAL(12,2) NOT NULL,
  data_vencimento   DATE          NOT NULL,
  status            ENUM('A_RECEBER','RECEBIDO_PARCIAL','RECEBIDO_TOTAL','EM_ATRASO')
                    NOT NULL DEFAULT 'A_RECEBER',
  id_pedido         INT UNSIGNED  NOT NULL,
  PRIMARY KEY (id_conta_receber),
  UNIQUE KEY uq_conta_receber_parcela (id_pedido, numero_parcela),
  KEY ix_conta_receber_vencimento (data_vencimento, status),
  CONSTRAINT fk_conta_receber_pedido FOREIGN KEY (id_pedido) REFERENCES pedido (id_pedido),
  CONSTRAINT ck_conta_receber_parcela CHECK (numero_parcela >= 1),
  CONSTRAINT ck_conta_receber_valor CHECK (valor_previsto > 0)
) ENGINE=InnoDB COMMENT='Direito de receber (uma linha por parcela)';

/* PAGAMENTO: várias linhas por conta = recebimentos parciais permitidos.
   id_taxa foi acrescentada (DER: PAGAMENTO utiliza TAXA_OPERADORA); é
   opcional porque dinheiro/Pix podem não ter taxa. */
CREATE TABLE pagamento (
  id_pagamento      INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  data_pagamento    DATE          NOT NULL,
  valor_recebido    DECIMAL(12,2) NOT NULL,
  metodo            VARCHAR(50)   NOT NULL,
  status            ENUM('PENDENTE','CONFIRMADO','ESTORNADO') NOT NULL DEFAULT 'PENDENTE',
  id_conta_receber  INT UNSIGNED  NOT NULL,
  id_taxa           INT UNSIGNED  NULL,
  PRIMARY KEY (id_pagamento),
  KEY ix_pagamento_conta (id_conta_receber),
  KEY ix_pagamento_taxa (id_taxa),
  KEY ix_pagamento_data (data_pagamento),
  CONSTRAINT fk_pagamento_conta FOREIGN KEY (id_conta_receber)
    REFERENCES conta_a_receber (id_conta_receber),
  CONSTRAINT fk_pagamento_taxa FOREIGN KEY (id_taxa) REFERENCES taxa_operadora (id_taxa),
  CONSTRAINT ck_pagamento_valor CHECK (valor_recebido > 0)
) ENGINE=InnoDB COMMENT='Recebimento efetivo de uma conta a receber';
```

```
/* ----------------------------------------------------------------------
   6. FINANCEIRO - PAGAMENTOS A FORNECEDORES E DESPESAS
   ---------------------------------------------------------------------- */

CREATE TABLE despesa (
  id_despesa      INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  descricao       VARCHAR(150)  NOT NULL,
  valor_previsto  DECIMAL(12,2) NOT NULL,
  competencia     DATE          NOT NULL,         -- 1º dia do mês de competência
  PRIMARY KEY (id_despesa),
  CONSTRAINT ck_despesa_valor CHECK (valor_previsto >= 0)
) ENGINE=InnoDB COMMENT='Gasto operacional (aluguel, energia, salários...)';

/* CONTA_A_PAGAR: PK corrigida (o script original escrevia IdcontasPagar).
   [DECISÃO] Origem exclusiva: compra OU despesa (DER liga a ambas; o
   documento pede para permitir despesas sem COMPRA). Direção da FK com
   DESPESA segue o DER (cada conta a pagar aponta para 1 despesa). */
CREATE TABLE conta_a_pagar (
  id_conta_pagar   INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  data_emissao     DATE          NOT NULL,
  data_vencimento  DATE          NOT NULL,
  valor_original   DECIMAL(12,2) NOT NULL,
  numero_parcela   TINYINT UNSIGNED NOT NULL DEFAULT 1,
  status           ENUM('ABERTO','PARCIAL','PAGO','ATRASADO') NOT NULL DEFAULT 'ABERTO',
  id_compra        INT UNSIGNED  NULL,
  id_despesa       INT UNSIGNED  NULL,
  PRIMARY KEY (id_conta_pagar),
  KEY ix_conta_pagar_vencimento (data_vencimento, status),
  KEY ix_conta_pagar_compra (id_compra),
  KEY ix_conta_pagar_despesa (id_despesa),
  CONSTRAINT fk_conta_pagar_compra FOREIGN KEY (id_compra) REFERENCES compra (id_compra),
  CONSTRAINT fk_conta_pagar_despesa FOREIGN KEY (id_despesa) REFERENCES despesa (id_despesa),
  CONSTRAINT ck_conta_pagar_origem CHECK ((id_compra IS NULL) <> (id_despesa IS NULL)),
  CONSTRAINT ck_conta_pagar_valor CHECK (valor_original > 0),
  CONSTRAINT ck_conta_pagar_parcela CHECK (numero_parcela >= 1),
  CONSTRAINT ck_conta_pagar_datas CHECK (data_vencimento >= data_emissao)
) ENGINE=InnoDB COMMENT='Obrigação de pagar (uma linha por parcela)';

/* BAIXA_PAGAR: várias baixas por conta = pagamentos parciais permitidos.
   FK corrigida para conta_a_pagar(id_conta_pagar). */
CREATE TABLE baixa_pagar (
  id_baixa        INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  data_baixa      DATE          NOT NULL,
  valor_pago      DECIMAL(12,2) NOT NULL,
  metodo          VARCHAR(50)   NOT NULL,
  id_conta_pagar  INT UNSIGNED  NOT NULL,
  PRIMARY KEY (id_baixa),
  KEY ix_baixa_conta (id_conta_pagar),
  KEY ix_baixa_data (data_baixa),
  CONSTRAINT fk_baixa_pagar_conta FOREIGN KEY (id_conta_pagar)
    REFERENCES conta_a_pagar (id_conta_pagar),
  CONSTRAINT ck_baixa_valor CHECK (valor_pago > 0)
) ENGINE=InnoDB COMMENT='Quitação (total ou parcial) de uma conta a pagar';

/* ----------------------------------------------------------------------
   7. CAIXA E CONCILIAÇÃO
   ---------------------------------------------------------------------- */

/* MOVIMENTO_CAIXA: registra só dinheiro efetivo. No DER recebe 1 PAGAMENTO
   (entrada) OU 1 BAIXA_PAGAR (saída). [DECISÃO] duas FKs opcionais, exatamente
   uma preenchida (CHECK) e coerente com o tipo; UNIQUE garante 1 movimento
   por pagamento/baixa. */
CREATE TABLE movimento_caixa (
  id_movimento_caixa  INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  data_movimento      DATE          NOT NULL,
  tipo                ENUM('ENTRADA','SAIDA') NOT NULL,
  valor_bruto         DECIMAL(12,2) NOT NULL,
  valor_liquido       DECIMAL(12,2) NOT NULL,
  id_pagamento        INT UNSIGNED  NULL,
  id_baixa            INT UNSIGNED  NULL,
  PRIMARY KEY (id_movimento_caixa),
  UNIQUE KEY uq_movcaixa_pagamento (id_pagamento),
  UNIQUE KEY uq_movcaixa_baixa (id_baixa),
  KEY ix_movcaixa_data (data_movimento),
  CONSTRAINT fk_movcaixa_pagamento FOREIGN KEY (id_pagamento) REFERENCES pagamento (id_pagamento),
  CONSTRAINT fk_movcaixa_baixa FOREIGN KEY (id_baixa) REFERENCES baixa_pagar (id_baixa),
  CONSTRAINT ck_movcaixa_origem CHECK ((id_pagamento IS NULL) <> (id_baixa IS NULL)),
  CONSTRAINT ck_movcaixa_tipo CHECK (
       (tipo = 'ENTRADA' AND id_pagamento IS NOT NULL)
    OR (tipo = 'SAIDA'   AND id_baixa     IS NOT NULL)),
  CONSTRAINT ck_movcaixa_valores CHECK (valor_bruto >= 0 AND valor_liquido >= 0
```

```
                                      AND valor_liquido <= valor_bruto)
) ENGINE=InnoDB COMMENT='Entrada/saída efetiva de dinheiro';

/* CONCILIACAO: confere o movimento de caixa com o extrato bancário.
   diferenca = valor_extrato - valor_esperado (coluna calculada).
   [DECISÃO] status_conciliacao calculado aplica a RN16:
   ABS(diferenca) > 0,05  =>  DIVERGENTE. Não consta no DER; adição mínima
   necessária para a RN16. */
CREATE TABLE conciliacao (
  id_conciliacao      INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  data_conciliacao    DATE          NOT NULL,
  valor_esperado      DECIMAL(12,2) NOT NULL,
  valor_extrato       DECIMAL(12,2) NOT NULL,
  diferenca           DECIMAL(12,2) GENERATED ALWAYS AS (valor_extrato - valor_esperado) STORED,
  status_conciliacao  VARCHAR(10)   GENERATED ALWAYS AS (
                        CASE WHEN ABS(valor_extrato - valor_esperado) > 0.05
                             THEN 'DIVERGENTE' ELSE 'CONCILIADO' END) STORED,
  id_movimento_caixa  INT UNSIGNED  NOT NULL,
  PRIMARY KEY (id_conciliacao),
  UNIQUE KEY uq_conciliacao_movimento (id_movimento_caixa),
  KEY ix_conciliacao_status (status_conciliacao),
  CONSTRAINT fk_conciliacao_movimento FOREIGN KEY (id_movimento_caixa)
    REFERENCES movimento_caixa (id_movimento_caixa)
) ENGINE=InnoDB COMMENT='Conferência entre o movimento e o extrato bancário (RN16)';
```

Fim do arquivo 01_schema(1).sql

APÊNDICE B — INSERÇÃO DE DADOS PARA TESTES

Arquivo original: 02_inserção_dados(1).sql | SGBD: MySQL 8.0+

```
/*
================================================================================
PATA FELIZ - Sistema de Gestão
Script 02 - Inserção de dados de teste (DML)
SGBD: MySQL 8.0+
Dependência: 01_schema.sql já executado
Banco: pata_feliz

IMPORTANTE:
- Este script insere dados fictícios para testes.
- Execute-o uma única vez em uma base recém-criada ou vazia.
- Os IDs são explícitos para facilitar a leitura e a validação das consultas.
- Não execute 01_schema.sql novamente depois de inserir estes dados:
  esse script contém DROP DATABASE IF EXISTS e apagará os registros.
- Se precisar recomeçar, execute 01_schema.sql e depois este arquivo.
================================================================================
*/

USE pata_feliz;

START TRANSACTION;

/* ------------------------------------------------------------------------
   1. CADASTROS BÁSICOS
   ------------------------------------------------------------------------ */

INSERT INTO tutor (id_tutor, nome, email, telefone, cpf) VALUES
(1, 'Mariana Costa', 'mariana.costa@email.com', '11987654321', '12345678901'),
(2, 'Rafael Almeida', 'rafael.almeida@email.com', '11987654322', '23456789012'),
(3, 'Camila Santos', 'camila.santos@email.com', '11987654323', NULL),
(4, 'Bruno Oliveira', 'bruno.oliveira@email.com', '11987654324', '45678901234'),
(5, 'Fernanda Lima', 'fernanda.lima@email.com', '11987654325', NULL);

INSERT INTO pet (id_pet, nome, raca, sexo) VALUES
(1, 'Mel', 'Golden Retriever', 'F'),
(2, 'Thor', 'Labrador', 'M'),
(3, 'Nina', 'Shih Tzu', 'F'),
(4, 'Max', 'Vira-lata', 'M'),
(5, 'Luna', 'Poodle', 'F'),
(6, 'Bob', 'Bulldog Francês', 'M');

INSERT INTO tutela (id_tutor, id_pet) VALUES
(1, 1),
(1, 2),
(2, 3),
(3, 4),
(4, 5),
(5, 6),
(2, 4); -- Exemplo de pet associado a mais de um tutor.

INSERT INTO funcionario (id_funcionario, nome, cpf, telefone, cargo) VALUES
(1, 'Ana Paula Ribeiro', '56789012345', '11990001111', 'Tosadora'),
(2, 'Carlos Mendes', '67890123456', '11990002222', 'Banhista'),
(3, 'Juliana Ferreira', '78901234567', '11990003333', 'Veterinaria'),
(4, 'Pedro Henrique Souza', '89012345678', '11990004444', 'Atendente');

INSERT INTO servico
(id_servico, nome, descricao, valor_base, duracao_estimada) VALUES
(1, 'Banho', 'Banho completo para pets', 65.00, 60),
(2, 'Tosa', 'Tosa higiênica e estética', 85.00, 90),
(3, 'Consulta', 'Consulta veterinária', 120.00, 45),
(4, 'Hidratação', 'Hidratação da pelagem', 45.00, 30),
(5, 'Corte de unhas', 'Corte e cuidado das unhas', 25.00, 15);

/* ------------------------------------------------------------------------
   2. AGENDAMENTOS E ATENDIMENTOS
   ------------------------------------------------------------------------ */

INSERT INTO agendamento
(id_agendamento, data_hora_prevista, status, id_tutor, id_pet, id_servico) VALUES
(1, '2026-11-03 09:00:00', 'CONCLUIDO', 1, 1, 1),
(2, '2026-11-04 10:30:00', 'CONCLUIDO', 2, 3, 2),
(3, '2026-11-08 14:00:00', 'AGENDADO', 3, 4, 3),
(4, '2026-11-09 11:00:00', 'CANCELADO', 4, 5, 1),
(5, '2026-11-10 15:00:00', 'REAGENDADO', 5, 6, 4),
(6, '2026-11-05 13:00:00', 'AUSENTE', 1, 2, 5);
```

```
INSERT INTO atendimento
(id_atendimento, data_hora_inicio, data_hora_fim, descricao, status,
 id_pet, id_funcionario, id_agendamento) VALUES
(1, '2026-11-03 09:05:00', '2026-11-03 10:00:00',
 'Banho completo realizado; pelagem em boas condições.',
 'CONCLUIDO', 1, 2, 1),
(2, '2026-11-04 10:35:00', '2026-11-04 12:00:00',
 'Tosa realizada conforme solicitação do tutor.',
 'CONCLUIDO', 3, 1, 2),
(3, '2026-11-06 16:00:00', '2026-11-06 16:40:00',
 'Atendimento realizado sem agendamento prévio.',
 'CONCLUIDO', 2, 3, NULL);

INSERT INTO item_atendimento
(id_item_atendimento, valor_cobrado, id_atendimento, id_servico) VALUES
(1, 65.00, 1, 1),
(2, 85.00, 2, 2),
(3, 120.00, 3, 3),
(4, 45.00, 1, 4);

/* ----------------------------------------------------------------------------
   3. PRODUTOS, ESTOQUE E FORNECEDORES
   ---------------------------------------------------------------------------- */

INSERT INTO produto
(id_produto, nome, descricao, quantidade_estoque, preco_venda_atual) VALUES
(1, 'Shampoo Neutro 500ml', 'Shampoo para higiene de pets', 40, 32.90),
(2, 'Condicionador 500ml', 'Condicionador para pelagem', 25, 38.50),
(3, 'Ração Premium 10kg', 'Ração para cães adultos', 18, 189.90),
(4, 'Brinquedo Mordedor', 'Brinquedo resistente para cães', 30, 29.90),
(5, 'Escova Pet', 'Escova para cuidados com a pelagem', 15, 24.90),
(6, 'Petisco Natural', 'Petisco para cães', 50, 18.00);

INSERT INTO fornecedor
(id_fornecedor, nome, cnpj, telefone, email) VALUES
(1, 'Distribuidora Animal', '12345678000190', '1133334444', 'vendas@distribuidoraan.com.br'),
(2, 'Pet Produtos Brasil', '23456789000101', '1144445555', 'contato@petprodutos.com.br'),
(3, 'Mundo Pet Atacado', '34567890000112', '1155556666', 'comercial@mundopet.com.br');

INSERT INTO produto_fornecedor
(id_produto, id_fornecedor, preco_referencia, prazo_entrega, codigo_produto_fornecedor) VALUES
(1, 1, 19.50, 5, 'SHN-500'),
(2, 1, 23.00, 5, 'CON-500'),
(3, 2, 142.00, 7, 'RAC-PRE-10'),
(4, 3, 15.00, 4, 'BRQ-MOR'),
(5, 2, 12.50, 6, 'ESC-PET'),
(6, 3, 9.00, 4, 'PTC-NAT'),
(3, 3, 145.00, 8, 'RAC-PRE-10B');

INSERT INTO movimentacao_estoque
(id_movimentacao, data_hora, tipo, motivo, quantidade, id_produto) VALUES
(1, '2026-10-25 09:00:00', 'ENTRADA', 'Estoque inicial', 50, 1),
(2, '2026-10-25 09:10:00', 'ENTRADA', 'Estoque inicial', 30, 2),
(3, '2026-10-25 09:20:00', 'ENTRADA', 'Estoque inicial', 20, 3),
(4, '2026-11-02 14:00:00', 'SAIDA', 'Venda de produtos', 10, 1),
(5, '2026-11-03 10:00:00', 'AJUSTE_POSITIVO', 'Correção de inventário', 2, 4),
(6, '2026-11-04 15:00:00', 'AJUSTE_NEGATIVO', 'Avaria identificada', 1, 5),
(7, '2026-10-25 09:30:00', 'ENTRADA', 'Estoque inicial', 28, 4),
(8, '2026-10-25 09:40:00', 'ENTRADA', 'Estoque inicial', 16, 5),
(9, '2026-10-25 09:50:00', 'ENTRADA', 'Estoque inicial', 50, 6);

/* ----------------------------------------------------------------------------
   4. PEDIDOS E COMPRAS
   ---------------------------------------------------------------------------- */

INSERT INTO pedido (id_pedido, data_pedido, status, id_tutor) VALUES
(1, '2026-11-01 10:15:00', 'CONFIRMADO', 1),
(2, '2026-11-02 11:20:00', 'CONFIRMADO', 2),
(3, '2026-11-03 15:30:00', 'CONFIRMADO', 1),
(4, '2026-11-05 09:45:00', 'ABERTO', 3),
(5, '2026-11-06 16:10:00', 'CANCELADO', 4);

INSERT INTO item_pedido
(id_pedido, id_produto, quantidade, preco_unitario) VALUES
(1, 1, 2, 32.90),
(1, 6, 3, 18.00),
(2, 3, 1, 189.90),
(2, 4, 2, 29.90),
(3, 2, 1, 38.50),
(3, 5, 1, 24.90),
```

```
(4, 6, 2, 18.00),
(5, 4, 1, 29.90);

INSERT INTO compra (id_compra, data_compra, status, id_fornecedor) VALUES
(1, '2026-10-25', 'RECEBIDA', 1),
(2, '2026-10-26', 'RECEBIDA', 2),
(3, '2026-11-06', 'ABERTA', 3);

INSERT INTO item_compra
(id_compra, id_produto, quantidade, preco_unitario_compra) VALUES
(1, 1, 50, 19.50),
(1, 2, 30, 23.00),
(2, 3, 20, 142.00),
(2, 5, 20, 12.50),
(3, 4, 40, 15.00),
(3, 6, 60, 9.00);

/* ------------------------------------------------------------------------
   5. FINANCEIRO - RECEBIMENTOS
   ------------------------------------------------------------------------ */

INSERT INTO taxa_operadora
(id_taxa, forma_pagamento, percentual_taxa, taxa_fixa) VALUES
(1, 'Cartao de credito', 2.50, 0.50),
(2, 'Cartao de debito', 1.20, 0.30),
(3, 'Pix', 0.00, 0.00);

INSERT INTO conta_a_receber
(id_conta_receber, numero_parcela, valor_previsto, data_vencimento, status, id_pedido) VALUES
(1, 1, 119.80, '2026-11-01', 'RECEBIDO_TOTAL', 1),
(2, 1, 249.70, '2026-11-02', 'RECEBIDO_PARCIAL', 2),
(3, 1, 63.40, '2026-11-03', 'A_RECEBER', 3),
(4, 1, 36.00, '2026-11-10', 'A_RECEBER', 4);

INSERT INTO pagamento
(id_pagamento, data_pagamento, valor_recebido, metodo, status,
 id_conta_receber, id_taxa) VALUES
(1, '2026-11-01', 119.80, 'Pix', 'CONFIRMADO', 1, 3),
(2, '2026-11-02', 150.00, 'Cartao de credito', 'CONFIRMADO', 2, 1),
(3, '2026-11-04', 50.00, 'Cartao de debito', 'CONFIRMADO', 2, 2);

/* ------------------------------------------------------------------------
   6. FINANCEIRO - DESPESAS, CONTAS A PAGAR E BAIXAS
   ------------------------------------------------------------------------ */

INSERT INTO despesa
(id_despesa, descricao, valor_previsto, competencia) VALUES
(1, 'Aluguel do estabelecimento', 2500.00, '2026-11-01'),
(2, 'Conta de energia', 480.00, '2026-11-01'),
(3, 'Serviço de internet', 150.00, '2026-11-01');

INSERT INTO conta_a_pagar
(id_conta_pagar, data_emissao, data_vencimento, valor_original,
 numero_parcela, status, id_compra, id_despesa) VALUES
(1, '2026-10-25', '2026-11-10', 1665.00, 1, 'PARCIAL', 1, NULL),
(2, '2026-10-26', '2026-11-12', 3090.00, 1, 'ABERTO', 2, NULL),
(3, '2026-11-01', '2026-11-05', 2500.00, 1, 'PARCIAL', NULL, 1),
(4, '2026-11-01', '2026-11-08', 480.00, 1, 'ABERTO', NULL, 2),
(5, '2026-11-01', '2026-11-15', 150.00, 1, 'ABERTO', NULL, 3);

INSERT INTO baixa_pagar
(id_baixa, data_baixa, valor_pago, metodo, id_conta_pagar) VALUES
(1, '2026-11-05', 500.00, 'Pix', 1),
(2, '2026-11-05', 1000.00, 'Transferencia', 3);

/* ------------------------------------------------------------------------
   7. MOVIMENTO DE CAIXA E CONCILIAÇÃO
   ------------------------------------------------------------------------ */

INSERT INTO movimento_caixa
(id_movimento_caixa, data_movimento, tipo, valor_bruto, valor_liquido,
 id_pagamento, id_baixa) VALUES
(1, '2026-11-01', 'ENTRADA', 119.80, 119.80, 1, NULL),
(2, '2026-11-02', 'ENTRADA', 150.00, 145.75, 2, NULL),
(3, '2026-11-04', 'ENTRADA', 50.00, 49.40, 3, NULL),
(4, '2026-11-05', 'SAIDA', 500.00, 500.00, NULL, 1),
(5, '2026-11-05', 'SAIDA', 1000.00, 1000.00, NULL, 2);

INSERT INTO conciliacao
(id_conciliacao, data_conciliacao, valor_esperado, valor_extrato, id_movimento_caixa) VALUES
```

```
(1, '2026-11-02', 119.80, 119.80, 1),
(2, '2026-11-03', 145.75, 145.75, 2),
(3, '2026-11-05', 49.40, 49.30, 3),
(4, '2026-11-06', 500.00, 500.00, 4);

COMMIT;

/* Conferência rápida: deve retornar a quantidade de registros por tabela. */
SELECT 'tutor' AS tabela, COUNT(*) AS total FROM tutor
UNION ALL SELECT 'pet', COUNT(*) FROM pet
UNION ALL SELECT 'tutela', COUNT(*) FROM tutela
UNION ALL SELECT 'funcionario', COUNT(*) FROM funcionario
UNION ALL SELECT 'servico', COUNT(*) FROM servico
UNION ALL SELECT 'agendamento', COUNT(*) FROM agendamento
UNION ALL SELECT 'atendimento', COUNT(*) FROM atendimento
UNION ALL SELECT 'item_atendimento', COUNT(*) FROM item_atendimento
UNION ALL SELECT 'produto', COUNT(*) FROM produto
UNION ALL SELECT 'movimentacao_estoque', COUNT(*) FROM movimentacao_estoque
UNION ALL SELECT 'fornecedor', COUNT(*) FROM fornecedor
UNION ALL SELECT 'produto_fornecedor', COUNT(*) FROM produto_fornecedor
UNION ALL SELECT 'pedido', COUNT(*) FROM pedido
UNION ALL SELECT 'item_pedido', COUNT(*) FROM item_pedido
UNION ALL SELECT 'compra', COUNT(*) FROM compra
UNION ALL SELECT 'item_compra', COUNT(*) FROM item_compra
UNION ALL SELECT 'taxa_operadora', COUNT(*) FROM taxa_operadora
UNION ALL SELECT 'conta_a_receber', COUNT(*) FROM conta_a_receber
UNION ALL SELECT 'pagamento', COUNT(*) FROM pagamento
UNION ALL SELECT 'despesa', COUNT(*) FROM despesa
UNION ALL SELECT 'conta_a_pagar', COUNT(*) FROM conta_a_pagar
UNION ALL SELECT 'baixa_pagar', COUNT(*) FROM baixa_pagar
UNION ALL SELECT 'movimento_caixa', COUNT(*) FROM movimento_caixa
UNION ALL SELECT 'conciliacao', COUNT(*) FROM conciliacao;
```

Fim do arquivo 02_inserção_dados(1).sql

APÊNDICE C — CONSULTAS GERENCIAIS E DE AUDITORIA

Arquivo original: 03_consultas(1).sql | SGBD: MySQL 8.0+

```
/*
================================================================================
PATA FELIZ - Sistema de Gestão
Script 03 - Consultas gerenciais e de auditoria (DQL)
SGBD: MySQL 8.0+
Dependência: 01_schema.sql e 02_dados_teste.sql já executados
Banco: pata_feliz

As consultas são independentes. Execute uma por vez ou selecione o bloco
desejado no MySQL Workbench.
================================================================================
*/

USE pata_feliz;



/* ============================================================================
   A. CADASTROS, TUTORES E PETS
   ============================================================================ */

/* 01. Listar todos os tutores. */

SELECT
    id_tutor,
    nome,
    email,
    telefone
FROM tutor
ORDER BY nome;



/* 02. Listar pets e seus tutores.
   Um pet pode aparecer mais de uma vez se possuir mais de um tutor em TUTELA. */

SELECT
    p.id_pet,
    p.nome AS pet,
    p.raca,
    p.sexo,
    t.id_tutor,
    t.nome AS tutor
FROM tutela tu
INNER JOIN tutor t ON t.id_tutor = tu.id_tutor
INNER JOIN pet p ON p.id_pet = tu.id_pet
ORDER BY p.nome, t.nome;



/* 03. Localizar pets que ainda não possuem vínculo em TUTELA. */

SELECT
    p.id_pet,
    p.nome,
    p.raca
FROM pet p
LEFT JOIN tutela tu ON tu.id_pet = p.id_pet
WHERE tu.id_pet IS NULL
ORDER BY p.nome;



/* ============================================================================
   B. AGENDAMENTOS E ATENDIMENTOS
   ============================================================================ */

/* 04. Agenda com tutor, pet e serviço. */

SELECT
    a.id_agendamento,
    a.data_hora_prevista,
    a.status,
    t.nome AS tutor,
    p.nome AS pet,
    s.nome AS servico
FROM agendamento a
INNER JOIN tutor t ON t.id_tutor = a.id_tutor
INNER JOIN pet p ON p.id_pet = a.id_pet
```

```sql
INNER JOIN servico s ON s.id_servico = a.id_servico
ORDER BY a.data_hora_prevista;


/* 05. Histórico de atendimentos de cada pet. */

SELECT
    p.nome AS pet,
    at.id_atendimento,
    at.data_hora_inicio,
    at.data_hora_fim,
    at.status,
    f.nome AS funcionario,
    at.descricao
FROM atendimento at
INNER JOIN pet p ON p.id_pet = at.id_pet
INNER JOIN funcionario f ON f.id_funcionario = at.id_funcionario
ORDER BY p.nome, at.data_hora_inicio DESC;


/* 06. Serviços executados e valores cobrados por atendimento. */

SELECT
    at.id_atendimento,
    p.nome AS pet,
    s.nome AS servico,
    ia.valor_cobrado,
    f.nome AS funcionario,
    at.data_hora_inicio
FROM item_atendimento ia
INNER JOIN atendimento at ON at.id_atendimento = ia.id_atendimento
INNER JOIN pet p ON p.id_pet = at.id_pet
INNER JOIN servico s ON s.id_servico = ia.id_servico
INNER JOIN funcionario f ON f.id_funcionario = at.id_funcionario
ORDER BY at.data_hora_inicio DESC, at.id_atendimento;


/* 07. Quantidade de atendimentos concluídos por funcionário. */

SELECT
    f.id_funcionario,
    f.nome AS funcionario,
    COUNT(at.id_atendimento) AS total_atendimentos_concluidos
FROM funcionario f
LEFT JOIN atendimento at
    ON at.id_funcionario = f.id_funcionario
   AND at.status = 'CONCLUIDO'
GROUP BY f.id_funcionario, f.nome
ORDER BY total_atendimentos_concluidos DESC, f.nome;


/* ============================================================================
   C. PRODUTOS, ESTOQUE E FORNECEDORES
   ============================================================================ */

/* 08. Consulta geral de produtos e estoque. */

SELECT
    id_produto,
    nome,
    quantidade_estoque,
    preco_venda_atual,
    quantidade_estoque * preco_venda_atual AS valor_estimado_estoque
FROM produto
ORDER BY quantidade_estoque ASC, nome;


/* 09. Produtos e seus fornecedores. */

SELECT
    p.nome AS produto,
    f.nome AS fornecedor,
    pf.preco_referencia,
    pf.prazo_entrega,
    pf.codigo_produto_fornecedor
FROM produto_fornecedor pf
INNER JOIN produto p ON p.id_produto = pf.id_produto
INNER JOIN fornecedor f ON f.id_fornecedor = pf.id_fornecedor
ORDER BY p.nome, f.nome;
```

```sql
/* 10. Histórico de movimentações de estoque por produto. */

SELECT
    p.nome AS produto,
    me.data_hora,
    me.tipo,
    me.quantidade,
    me.motivo
FROM movimentacao_estoque me
INNER JOIN produto p ON p.id_produto = me.id_produto
ORDER BY p.nome, me.data_hora DESC;



/* ============================================================================
   D. PEDIDOS E COMPRAS
   ============================================================================ */

/* 11. Pedidos realizados por tutor, com seus itens.
   O subtotal é calculado: quantidade x preço unitário. */

SELECT
    pe.id_pedido,
    pe.data_pedido,
    pe.status,
    t.nome AS tutor,
    p.nome AS produto,
    ip.quantidade,
    ip.preco_unitario,
    (ip.quantidade * ip.preco_unitario) AS subtotal_item
FROM pedido pe
INNER JOIN tutor t ON t.id_tutor = pe.id_tutor
INNER JOIN item_pedido ip ON ip.id_pedido = pe.id_pedido
INNER JOIN produto p ON p.id_produto = ip.id_produto
ORDER BY pe.data_pedido DESC, pe.id_pedido, p.nome;



/* 12. Valor total dos itens de cada pedido.
   O resultado é calculado a partir dos itens, não de uma coluna total em PEDIDO. */

SELECT
    pe.id_pedido,
    pe.data_pedido,
    pe.status,
    t.nome AS tutor,
    COALESCE(SUM(ip.quantidade * ip.preco_unitario), 0) AS total_pedido
FROM pedido pe
INNER JOIN tutor t ON t.id_tutor = pe.id_tutor
LEFT JOIN item_pedido ip ON ip.id_pedido = pe.id_pedido
GROUP BY pe.id_pedido, pe.data_pedido, pe.status, t.nome
ORDER BY pe.data_pedido DESC;



/* 13. Produtos mais vendidos por quantidade.
   Considera apenas pedidos que não estejam cancelados. */

SELECT
    p.id_produto,
    p.nome AS produto,
    COALESCE(SUM(
        CASE
            WHEN pe.status <> 'CANCELADO'
            THEN ip.quantidade
            ELSE 0
        END
    ), 0) AS quantidade_vendida
FROM produto p
LEFT JOIN item_pedido ip ON ip.id_produto = p.id_produto
LEFT JOIN pedido pe ON pe.id_pedido = ip.id_pedido
GROUP BY p.id_produto, p.nome
ORDER BY quantidade_vendida DESC, p.nome;



/* 14. Compras realizadas por fornecedor e respectivos itens. */

SELECT
    c.id_compra,
    c.data_compra,
```

```sql
    c.status,
    f.nome AS fornecedor,
    p.nome AS produto,
    ic.quantidade,
    ic.preco_unitario_compra,
    (ic.quantidade * ic.preco_unitario_compra) AS subtotal_compra
FROM compra c
INNER JOIN fornecedor f ON f.id_fornecedor = c.id_fornecedor
INNER JOIN item_compra ic ON ic.id_compra = c.id_compra
INNER JOIN produto p ON p.id_produto = ic.id_produto
ORDER BY c.data_compra DESC, c.id_compra;



/* ========================================================================
   E. FINANCEIRO - CONTAS A RECEBER E PAGAMENTOS
   ======================================================================== */

/* 15. Contas a receber com tutor e pedido de origem. */

SELECT
    car.id_conta_receber,
    car.numero_parcela,
    car.valor_previsto,
    car.data_vencimento,
    car.status,
    pe.id_pedido,
    t.nome AS tutor
FROM conta_a_receber car
INNER JOIN pedido pe ON pe.id_pedido = car.id_pedido
INNER JOIN tutor t ON t.id_tutor = pe.id_tutor
ORDER BY car.data_vencimento;



/* 16. Total recebido por conta e saldo calculado.
   Considera apenas pagamentos CONFIRMADOS. */

SELECT
    car.id_conta_receber,
    car.valor_previsto,
    COALESCE(SUM(
        CASE
            WHEN pg.status = 'CONFIRMADO'
            THEN pg.valor_recebido
            ELSE 0
        END
    ), 0) AS total_recebido_confirmado,

    car.valor_previsto - COALESCE(SUM(
        CASE
            WHEN pg.status = 'CONFIRMADO'
            THEN pg.valor_recebido
            ELSE 0
        END
    ), 0) AS saldo_calculado

FROM conta_a_receber car
LEFT JOIN pagamento pg
    ON pg.id_conta_receber = car.id_conta_receber
GROUP BY car.id_conta_receber, car.valor_previsto
ORDER BY car.id_conta_receber;



/* 17. Pagamentos recebidos por período.
   Altere as datas do WHERE conforme o período desejado. */

SELECT
    pg.data_pagamento,
    pg.metodo,
    COUNT(*) AS quantidade_pagamentos,
    SUM(pg.valor_recebido) AS total_registrado
FROM pagamento pg
WHERE pg.status = 'CONFIRMADO'
  AND pg.data_pagamento BETWEEN '2026-11-01' AND '2026-11-30'
GROUP BY pg.data_pagamento, pg.metodo
ORDER BY pg.data_pagamento, pg.metodo;



/* ========================================================================
   F. FINANCEIRO - CONTAS A PAGAR E DESPESAS
   ======================================================================== */
```

```sql
/* 18. Contas a pagar e sua origem (compra ou despesa). */

SELECT
    cap.id_conta_pagar,
    cap.data_emissao,
    cap.data_vencimento,
    cap.valor_original,
    cap.status,
    c.id_compra,
    d.descricao AS despesa
FROM conta_a_pagar cap
LEFT JOIN compra c ON c.id_compra = cap.id_compra
LEFT JOIN despesa d ON d.id_despesa = cap.id_despesa
ORDER BY cap.data_vencimento;


/* 19. Total baixado e saldo calculado por conta a pagar. */

SELECT
    cap.id_conta_pagar,
    cap.valor_original,
    COALESCE(SUM(bp.valor_pago), 0) AS total_baixado,
    cap.valor_original - COALESCE(SUM(bp.valor_pago), 0) AS saldo_calculado
FROM conta_a_pagar cap
LEFT JOIN baixa_pagar bp
    ON bp.id_conta_pagar = cap.id_conta_pagar
GROUP BY cap.id_conta_pagar, cap.valor_original
ORDER BY cap.id_conta_pagar;


/* ============================================================================
   G. CAIXA E CONCILIAÇÃO
   ============================================================================ */

/* 20. Movimentações de caixa e sua origem. */

SELECT
    mc.id_movimento_caixa,
    mc.data_movimento,
    mc.tipo,
    mc.valor_bruto,
    mc.valor_liquido,
    mc.id_pagamento,
    mc.id_baixa
FROM movimento_caixa mc
ORDER BY mc.data_movimento, mc.id_movimento_caixa;


/* 21. Resultado das conciliações. */

SELECT
    c.id_conciliacao,
    c.data_conciliacao,
    mc.tipo AS tipo_movimento,
    c.valor_esperado,
    c.valor_extrato,
    c.diferenca,
    c.status_conciliacao
FROM conciliacao c
INNER JOIN movimento_caixa mc
    ON mc.id_movimento_caixa = c.id_movimento_caixa
ORDER BY c.data_conciliacao DESC;


/* 22. Somente conciliações divergentes. */

SELECT
    id_conciliacao,
    data_conciliacao,
    valor_esperado,
    valor_extrato,
    diferenca,
    status_conciliacao
FROM conciliacao
WHERE status_conciliacao = 'DIVERGENTE'
ORDER BY data_conciliacao DESC;
```

```sql
/* ========================================================================
   H. CONSULTAS DE AUDITORIA E INTEGRIDADE OPERACIONAL
   ======================================================================== */

/* 23. Pedidos sem itens.
   A consulta identifica pedidos sem item; a RN05 exige tratamento adicional
   por trigger/procedure ou pela aplicação, conforme decisão do projeto. */

SELECT
    pe.id_pedido,
    pe.data_pedido,
    pe.status
FROM pedido pe
LEFT JOIN item_pedido ip ON ip.id_pedido = pe.id_pedido
WHERE ip.id_pedido IS NULL;


/* 24. Contas a receber sem pagamentos registrados. */

SELECT
    car.id_conta_receber,
    car.valor_previsto,
    car.data_vencimento,
    car.status
FROM conta_a_receber car
LEFT JOIN pagamento pg
    ON pg.id_conta_receber = car.id_conta_receber
WHERE pg.id_pagamento IS NULL;


/* 25. Contas a pagar sem baixas registradas. */

SELECT
    cap.id_conta_pagar,
    cap.valor_original,
    cap.data_vencimento,
    cap.status
FROM conta_a_pagar cap
LEFT JOIN baixa_pagar bp
    ON bp.id_conta_pagar = cap.id_conta_pagar
WHERE bp.id_baixa IS NULL;


/* 26. Movimentos de caixa ainda sem conciliação. */

SELECT
    mc.id_movimento_caixa,
    mc.data_movimento,
    mc.tipo,
    mc.valor_bruto,
    mc.valor_liquido
FROM movimento_caixa mc
LEFT JOIN conciliacao c
    ON c.id_movimento_caixa = mc.id_movimento_caixa
WHERE c.id_conciliacao IS NULL;


/* 27. Produtos com estoque zerado. */

SELECT
    id_produto,
    nome,
    quantidade_estoque
FROM produto
WHERE quantidade_estoque = 0
ORDER BY nome;


/* 28. Agendamentos que não foram concluídos.
   Útil para acompanhar cancelamentos, ausências e agendamentos futuros. */

SELECT
    a.id_agendamento,
    a.data_hora_prevista,
    a.status,
    t.nome AS tutor,
    p.nome AS pet,
    s.nome AS servico
FROM agendamento a
```

```
INNER JOIN tutor t ON t.id_tutor = a.id_tutor
INNER JOIN pet p ON p.id_pet = a.id_pet
INNER JOIN servico s ON s.id_servico = a.id_servico
WHERE a.status <> 'CONCLUIDO'
ORDER BY a.data_hora_prevista;
```

Fim do arquivo 03_consultas(1).sql